Modul 9 HMVC dan Laravel dengan Fortify

Panduan lengkap autentikasi Fortify dan CRUD dengan penjelasan setiap langkah

Modul ini membimbing mahasiswa membangun CRUD pendaftaran berbasis modul, membandingkan form biasa dengan AJAX, lalu membangun CRUD biodata React melalui Inertia. Setiap tahap menghubungkan tindakan yang dikerjakan dengan alasan, proses, hasil, dan cara memeriksanya.

Cara belajar dengan modul ini

Siapkan editor, terminal, browser, dan pengelola database. Buka folder proyek sebagai satu workspace agar pencarian berkas tidak berpindah ke proyek lain. Saat mengerjakan React, tutup workspace HMVC atau beri nama terminal yang jelas.

Untuk setiap langkah, baca tujuan lebih dahulu, kerjakan tindakan, lalu bandingkan hasil. Jika pemeriksaan gagal, hentikan penambahan kode dan perbaiki tahap itu. Error pada koneksi database dapat membuat banyak langkah berikutnya tampak salah, meskipun kode tampilan benar.

Gunakan satu akun dan data rekaan untuk latihan. Catat perubahan pada berkas dan hasil uji. Ukuran keberhasilan bukan banyaknya kode yang ditempel, melainkan kemampuan menjelaskan input, proses, output, dan alasan suatu kesalahan terjadi.

Hasil belajar

Setelah praktik, Anda mampu menunjukkan enam endpoint CRUD, membedakan GET dan POST, menjelaskan fungsi migration serta fillable, menolak input tidak valid, memeriksa login di server, dan membaca status HTTP dari Network.

Anda juga mampu menelusuri satu proses tambah dan satu proses edit pada tiga antarmuka. Pada CRUD biasa, buktinya navigasi dokumen; pada AJAX, buktinya JSON dan perubahan DOM; pada Inertia, buktinya props halaman serta redirect yang ditangani klien Inertia.

Lingkungan acuan

Laravel 12, nwidart/laravel-modules 12, Fortify 1.30 atau versi kompatibel dalam mayor 1, Composer 2, dan PHP 8.2 atau lebih baru yang kompatibel. Proyek HMVC memakai Fortify manual dengan form Blade. Proyek React memakai snapshot starter kit resmi Laravel 12 yang sudah memuat Fortify dan Inertia 2; Node 22.12 atau lebih baru dalam lini yang didukung Vite 7 diperlukan pada jalur React.

Prasyarat: penggunaan folder dan terminal, HTML form, dasar class PHP, SQL, serta event JavaScript. Gunakan data rekaan selama praktik. Dua proyek terpisah bernama praktikum-hmvc dan praktikum-react digunakan agar scaffolding tidak saling mengganti berkas.

Peta isi dan urutan pengerjaan

Klik judul untuk berpindah ke bagian terkait. Di Word, Navigation Pane juga menampilkan susunan Heading. Nomor di depan judul menunjukkan urutan bagian, bukan nomor halaman.

1 Memahami MVC dan modul

2 CRUD biasa AJAX dan Inertia

3 Menyiapkan Laravel dan Fortify

3A Memahami konfigurasi Fortify

3B Menghubungkan provider dan action registrasi

3C Membuat layout dan halaman login

3D Membuat registrasi dan dashboard

3E Menguji Fortify sebelum membuat CRUD

4 Menjalankan HMVC dengan sesi Fortify

5 Memasang dan mengenali modul

6 Membuat tabel dan model

7 Menentukan route yang dilindungi login

8 Controller untuk membaca dan menambah

9 Controller untuk mengubah dan menghapus

10 Membuat layout Blade

11 Membuat halaman daftar Blade

12 Membuat satu form untuk tambah dan edit

13 Memeriksa CRUD biasa

14 Menyiapkan tampilan AJAX

15 Mengirim request dan membaca error

16 Menggambar tabel dan menyimpan

17 Edit hapus dan pemeriksaan AJAX

18 Menyiapkan React Starter Kit dengan Fortify

19 Database dan model biodata

20 Controller biodata bagian pertama

21 Controller dan route biodata

22 Halaman daftar React

23 Form React untuk tambah dan edit

24 Memeriksa React dan autentikasi Fortify
25 Menemukan penyebab error
26 Tugas Modul 9 dan rubrik
27 Panduan dosen dan referensi
Lampiran A Glosarium istilah
Lampiran B Menelusuri satu data dari awal sampai akhir
Lampiran C Contoh README praktikum
Lampiran D Lembar pengujian

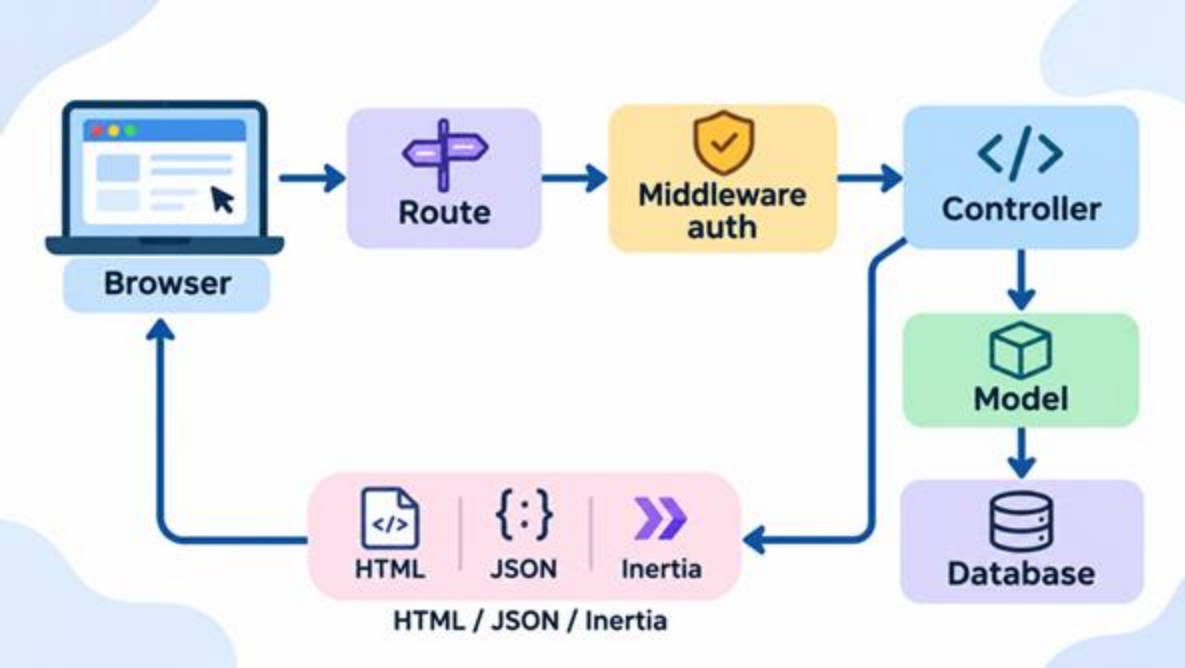
1 Memahami MVC dan modul

Satu fitur tersusun dari beberapa tanggung jawab.

Petunjuk pengerjaan dan kode

Model menghubungkan aplikasi dengan data. Daftar mewakili tabel daftars. Model tidak menggambar tabel HTML. View menampilkan informasi dan form. Blade dirender menjadi HTML di server, sedangkan React menggambar antarmuka melalui komponen di browser.

Controller menerima request, memvalidasi input, memanggil model, dan menentukan respons. Route memetakan alamat dan metode HTTP ke fungsi controller. Middleware auth memeriksa login sebelum controller dijalankan.



Pendekatan modular mengumpulkan berkas fitur dalam satu folder. Pendaftaran berada dalam Modules/Pendaftaran; fitur Keuangan dapat menjadi modul lain. Ini membantu pengelompokan fitur dan pembagian pekerjaan.

HMVC adalah istilah arsitektur yang lebih luas daripada memindahkan folder. Paket nwidart membantu membangun struktur modular. Praktikum ini memakai MVC modular yang umum disebut HMVC dalam pembelajaran, tanpa mendemonstrasikan pemanggilan controller bertingkat.

Contoh konkret: Rani mengisi form. Route POST memilih store. Controller memvalidasi nama, asal sekolah, dan prodi. Model menjalankan penyimpanan. Controller mengirim respons. Browser menampilkan hasil.

Cek pemahaman: apakah view boleh langsung menjalankan INSERT SQL? Dalam pola praktik ini, penyimpanan melewati route, controller, lalu model. Cobalah menunjuk berkas yang bertanggung jawab pada setiap langkah.

Membaca alur dari kebutuhan ke berkas

Bayangkan loket pendaftaran. Form adalah lembar yang diisi pemohon. Route menentukan loket tujuan berdasarkan alamat dan jenis permintaan. Controller memeriksa kelengkapan, model mengurus penyimpanan, dan database menyimpan catatan. Analogi ini membantu mengingat tanggung jawab, tetapi implementasinya tetap berupa request dan respons HTTP.

Ketika browser membuka GET /pendaftaran, tidak ada data baru yang disimpan. Controller index membaca model lalu mengirim daftar. Ketika form dikirim melalui POST /pendaftaran, controller store memvalidasi dan menyimpan. URL boleh sama karena metode HTTP membuat kedua route berbeda.

Mengapa berkas dikelompokkan menjadi modul

Dalam proyek besar, folder controller dapat berisi banyak fitur yang tidak berkaitan. Modul Pendaftaran mengelompokkan route, controller, model, migration, dan view yang berkaitan dengan pendaftaran. Ketika memperbaiki fitur itu, mahasiswa mengetahui area berkas yang perlu ditelusuri.

Database tetap dapat dipakai bersama oleh beberapa modul. Membuat modul tidak otomatis membuat database baru, server baru, atau layanan terpisah. Jangan memindahkan berkas secara manual sebelum memahami mapping namespace dan provider yang dibuat generator.

Latihan penelusuran

Sebutkan urutan saat pengguna menghapus ID 7. Jawaban yang diharapkan: browser mengirim DELETE, route cocok, auth memeriksa sesi, binding mencari model ID 7, destroy menghapus model, lalu respons memperbarui tampilan. Jika ID tidak ada, proses berhenti dengan 404 sebelum delete dijalankan.

2 CRUD biasa AJAX dan Inertia

Arsitektur dan cara memperbarui halaman adalah dua hal berbeda.

Petunjuk pengerjaan dan kode

CRUD berarti Create atau tambah, Read atau baca, Update atau ubah, dan Delete atau hapus. Operasi yang sama dapat disajikan melalui Blade, AJAX, maupun React.

CRUD biasa: browser mengirim form, server memproses, lalu mengarahkan browser ke daftar. Dokumen halaman dimuat kembali. Istilah synchronous pada tugas berarti pola navigasi halaman ini, bukan bahwa server hanya mampu mengerjakan satu request.

AJAX: JavaScript mengirim request dengan fetch, menerima JSON, dan memperbarui bagian halaman. Server serta database tetap bekerja. Tanpa refresh penuh tidak berarti data hanya disimpan di browser.

Inertia: controller mengirim nama halaman dan props ke React. Navigasi berlangsung melalui request di belakang layar. Contoh ini tidak membutuhkan REST API terpisah untuk tiap halaman.

```
Operasi     Metode    Contoh URL
Daftar      GET       /pendaftaran
Tambah      POST      /pendaftaran
Form edit   GET       /pendaftaran/7/edit
Ubah        PUT       /pendaftaran/7
Hapus       DELETE    /pendaftaran/7
```

GET dipakai membaca. Form HTML mendukung GET dan POST secara langsung. Blade menggunakan @method untuk PUT atau DELETE. fetch dan Inertia dapat mengirim metode tersebut langsung.

Peta praktik: tahap A menyiapkan login, tabel, route, controller, dan Blade. Tahap B menggunakan data serta controller yang sama dengan fetch. Tahap C menggunakan proyek React terpisah agar perbedaan alur terlihat.

Prediksi: jika data sudah ada di database tetapi tabel AJAX kosong, periksa request GET dan proses menggambar baris sebelum mengubah migration.

Mengenali bagian request dan respons

Request memiliki metode, URL, header, dan kadang body. Pada tambah, body membawa nama, asal_sekolah, dan prodi_tujuan. Header Accept menyatakan bentuk respons yang diinginkan, sedangkan Content-Type menjelaskan format body yang dikirim. Keduanya berbeda fungsi.

Respons memiliki status, header, dan body. Status 200 menunjukkan permintaan berhasil, 201 menunjukkan sumber daya baru dibuat, 302 atau 303 dapat dipakai dalam alur redirect, 401 menunjukkan belum terautentikasi pada request JSON, 404 menunjukkan sumber daya tidak ditemukan, dan 422 menunjukkan input tidak lolos validasi.

Membandingkan tiga mekanisme pembaruan

Pada Blade biasa, hasil validasi gagal dikembalikan melalui redirect dan error sesi, lalu halaman form digambar ulang. Pada fetch, request yang meminta JSON menerima rincian errors untuk dibaca JavaScript. Pada Inertia, integrasi redirect dan props error dikelola oleh useForm. Jangan memakai penanganan 422 fetch sebagai salinan langsung untuk semua form Inertia.

AJAX dan Inertia tetap melakukan komunikasi jaringan. Tanpa refresh penuh tidak sama dengan tanpa waktu tunggu. Karena itu antarmuka harus menunjukkan proses berjalan, mencegah klik berulang, dan menyampaikan kegagalan kepada pengguna.

Pemeriksaan sebelum praktik

Ambil selembar kertas dan tuliskan metode untuk baca, tambah, ubah, dan hapus. Jika masih tertukar, kembali ke tabel metode sebelum membuat route. Pada update, tentukan juga dari mana ID data diperoleh.

3 Menyiapkan Laravel dan Fortify

Mulai dari proyek HMVC baru agar endpoint autentikasi tidak bertabrakan.

Petunjuk pengerjaan dan kode

Gunakan folder baru praktikum-hmvc. Jika sudah membuat proyek latihan sebelumnya, pertahankan sebagai referensi dan gunakan nama folder baru untuk versi Fortify. Jangan menempel dua scaffolding autentikasi pada satu proyek.

```
php -v
composer --version
composer create-project laravel/laravel praktikum-hmvc "~12.0"
cd praktikum-hmvc
composer require laravel/fortify:"~1.30"
php artisan fortify:install
```

Siapkan .env. Bila belum ada, salin .env.example dan jalankan key:generate. Buat database praktikum_hmvc pada MySQL terlebih dahulu, kemudian sesuaikan username dan password.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=praktikum_hmvc
DB_USERNAME=root
DB_PASSWORD=
APP_URL=http://127.0.0.1:8000


php artisan config:clear
php artisan migrate
```

Lanjutkan bagian 3A sampai 3E untuk menyediakan konfigurasi, provider, action, view, dan dashboard. Fortify adalah backend autentikasi, sehingga halaman login tidak otomatis selesai hanya dengan composer require.

Langkah 1 Memeriksa alat

Jalankan php -v dan composer --version dari terminal. PHP menjalankan kode backend; Composer mengelola paket PHP. Laravel 12 memerlukan PHP yang kompatibel dan ekstensi yang diminta Composer, termasuk pdo_mysql untuk MySQL. Periksa pesan persyaratan sebelum melanjutkan.

Proyek HMVC menggunakan Blade dan JavaScript biasa dengan CSS langsung sehingga tidak memerlukan build frontend untuk contoh ini. Node dan npm baru wajib pada proyek React. Di sana gunakan Node 22.12 atau lebih baru dalam lini yang didukung Vite 7.

Langkah 2 Membuat aplikasi dan memasang Fortify

create-project membuat aplikasi Laravel 12 kosong. cd memindahkan terminal ke folder yang memiliki artisan. composer require laravel/fortify menambahkan backend autentikasi, sedangkan fortify:install menyalin konfigurasi, action, dan provider aplikasi.

Fortify tidak menghasilkan tampilan login dan register yang siap dipakai. Karena itu bagian 3A sampai 3E melengkapi konfigurasi serta tampilan Blade. Jangan menganggap keberhasilan instalasi paket berarti halaman /login sudah dapat dirender.

Langkah 3 Menghubungkan database

Buat database praktikum_hmvc terlebih dahulu lalu ubah .env. DB_HOST dan DB_PORT menunjuk layanan, DB_DATABASE memilih database, sedangkan username/password dipakai untuk koneksi. Jika .env masih memakai SQLite, ganti menjadi MySQL dan hindari variabel DB ganda.

config:clear membuat konfigurasi dibaca kembali. migrate menjalankan perubahan tabel yang belum dicatat. Periksa users, sessions bila session memakai database, dan migrations sebelum mencoba register. Jangan mengganti APP_KEY yang sudah digunakan; buat hanya jika belum ada.

3A Memahami konfigurasi Fortify

Fortify menangani backend autentikasi dan aplikasi menyediakan tampilannya.

Petunjuk pengerjaan dan kode

Kerjakan bagian 3A sampai 3E setelah instalasi Fortify dan database pada bagian 3 selesai. Gunakan proyek praktikum-hmvc. Konfigurasi dalam bagian ini tidak ditempel ke proyek React starter kit.

Buka bootstrap/providers.php. Pastikan provider aplikasi berikut tercantum satu kali. fortify:install pada versi yang mendukungnya dapat menambahkan provider secara otomatis. Jika belum ada, tambahkan dengan mempertahankan provider lain.

```
<?php
return [
    App\Providers\AppServiceProvider::class,
    App\Providers\FortifyServiceProvider::class,
];
```

Provider aplikasi berbeda dari provider paket yang ditemukan Composer. Provider aplikasi menghubungkan action dan view buatan Anda ke Fortify. Tanpanya, paket mungkin memiliki route tetapi tidak mengetahui tampilan login yang harus dipakai.

Untuk proyek HMVC baru ini, ganti config/fortify.php dengan konfigurasi inti berikut. Fitur yang diaktifkan hanya registrasi; login dan logout disediakan Fortify sebagai alur inti.

```
<?php
use Laravel\Fortify\Features;


return [
    'guard' => 'web',
    'passwords' => 'users',
    'username' => 'email',
    'email' => 'email',
    'lowercase_usernames' => true,
    'home' => '/dashboard',
    'prefix' => '',
    'domain' => null,
    'middleware' => ['web'],
    'views' => true,
    'limiters' => ['login' => 'login'],
    'features' => [Features::registration()],
];
```

guard web memakai autentikasi berbasis sesi. username email berarti form login harus memiliki name email. home dashboard menentukan tujuan sukses ketika tidak ada URL tujuan sebelumnya. views true mempertahankan route GET untuk halaman login dan register; Inertia juga dapat memakai views true.

middleware web mengaktifkan kebutuhan sesi dan CSRF pada endpoint. limiters login menunjuk pembatas bernama login yang dibuat di provider. Jangan mengaktifkan reset password, verifikasi email, atau 2FA sebelum tampilan dan alur pendukungnya disiapkan.

```
php artisan config:clear
```

Titik pemeriksaan: konfigurasi dapat dibaca tanpa error PHP. /login belum perlu dibuka sampai provider dan view selesai dibuat.

3B Menghubungkan provider dan action registrasi

Provider menentukan tampilan dan cara membuat pengguna.

Petunjuk pengerjaan dan kode

Pada proyek HMVC baru, ganti app/Providers/FortifyServiceProvider.php dengan kode berikut. Ini provider untuk fitur inti yang dipakai modul. Simpan berkas provider bawaan terlebih dahulu jika Anda sudah mengembangkan fitur lain.

```
<?php
namespace App\Providers;


use App\Actions\Fortify\CreateNewUser;
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\ServiceProvider;
use Laravel\Fortify\Fortify;


class FortifyServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Fortify::createUsersUsing(CreateNewUser::class);
        Fortify::loginView(fn () => view('auth.login'));
        Fortify::registerView(fn () => view('auth.register'));


        RateLimiter::for('login', function (Request $request) {
            $key = strtolower((string) $request->input('email'))
                .'|'.$request->ip();
            return Limit::perMinute(5)->by($key);
        });
    }
}
```

createUsersUsing menunjuk class yang akan membuat akun saat POST register berhasil divalidasi. loginView dan registerView mengembalikan view Blade. Ketiga baris ini menghubungkan fitur paket dengan kode aplikasi; Anda tidak menulis controller login sendiri.

RateLimiter membatasi percobaan login menggunakan kombinasi email dan IP. perMinute 5 adalah batas percobaan pada kunci itu. Email diubah ke huruf kecil agar perbedaan kapitalisasi tidak membuat kunci berbeda. Jangan menonaktifkan pembatas hanya karena pengujian password salah mencapai batas; tunggu masa pembatas berakhir.

Ganti app/Actions/Fortify/CreateNewUser.php dengan contoh berikut. Action ini khusus pembuatan akun, bukan untuk data pendaftaran mahasiswa.

```
<?php
namespace App\Actions\Fortify;


use App\Models\User;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Validator;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;
use Laravel\Fortify\Contracts\CreatesNewUsers;


class CreateNewUser implements CreatesNewUsers
{
    public function create(array $input): User
    {
        $input['email'] = strtolower(trim((string) ($input['email'] ?? '')));
        Validator::make($input, [
            'name' => ['required', 'string', 'max:100'],
            'email' => ['required', 'string', 'email', 'max:255',
                Rule::unique(User::class)],
            'password' => ['required', 'string', 'confirmed', Password::min(8)],
        ])->validate();


        return User::create([
            'name' => $input['name'],
            'email' => $input['email'],
            'password' => Hash::make($input['password']),
        ]);
    }
}
```

Validator memeriksa nama, format dan keunikan email, serta password. confirmed membandingkan password dengan password_confirmation. Email dinormalisasi sebelum validasi keunikan. Hash::make menyimpan hash, bukan password asli. Kolom users harus memakai fillable name, email, dan password sebagaimana model bawaan Laravel.

CreatesNewUsers adalah kontrak yang memberi tahu Fortify bentuk action yang diharapkan. create menerima array input dan mengembalikan User. Jika validasi gagal, User::create tidak dijalankan. Password tidak boleh diisi kembali melalui old pada form.

Titik pemeriksaan: ketiga berkas config, provider, dan action tersimpan pada lokasi yang benar. Nama class CreateNewUser sesuai dengan import di provider.

3C Membuat layout dan halaman login

Fortify menyediakan endpoint login sementara Blade menyediakan form.

Petunjuk pengerjaan dan kode

Buat folder resources/views/auth di proyek HMVC. Buat file layout.blade.php di dalamnya lalu isi berikut. CSS langsung membuat halaman latihan bisa dibuka tanpa menjalankan Vite.

```
<!doctype html>
<html lang="id">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Autentikasi Fortify</title>
    <style>
        body {font:16px Arial;max-width:720px;margin:32px auto;
              padding:0 16px;line-height:1.6}
        input:not([type=checkbox]) {display:block;padding:8px;
              margin:4px 0 12px;max-width:100%;box-sizing:border-box}
        .error {color:#b91c1c} nav {margin-bottom:20px}
    </style>
</head>
<body>
    <nav>
        @guest
            <a href="{{ url('/login') }}">Login</a>
            <a href="{{ url('/register') }}">Daftar akun</a>
        @endguest
        @auth
            <a href="{{ url('/dashboard') }}">Dashboard</a>
            <form method="POST" action="{{ url('/logout') }}"
                  style="display:inline">
                @csrf <button>Logout</button>
            </form>
        @endauth
    </nav>
    @yield('content')
</body>
</html>
```

guest menampilkan navigasi untuk pengguna belum login. auth menampilkan dashboard serta form logout untuk pengguna login. Logout menggunakan POST dan @csrf karena mengubah keadaan sesi; jangan membuat tautan GET logout.

Buat resources/views/auth/login.blade.php dengan isi berikut.

```
@extends('auth.layout')
@section('content')
```

```
<h1>Login</h1>
<form method="POST" action="{{ url('/login') }}">
    @csrf
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required autofocus
        autocomplete="username" value="{{ old('email') }}">
    @error('email') <p class="error">{{ $message }}</p> @enderror
    <label for="password">Password</label>
    <input id="password" name="password" type="password" required
        autocomplete="current-password">
    @error('password') <p class="error">{{ $message }}</p> @enderror
    <label><input type="checkbox" name="remember" value="1">
        Ingat saya pada perangkat ini</label>
    <button>Masuk</button>
</form>
<p>Belum punya akun? <a href="{{ url('/register') }}">Daftar</a></p>
@endsection
```

Form POST ke /login diproses Fortify. name email harus cocok dengan username pada konfigurasi. name password membawa password. remember bernilai 1 jika dicentang dan tidak dikirim jika tidak dicentang. Jangan memakai opsi ingat saya pada komputer bersama untuk demonstrasi kelas.

old email mempertahankan alamat ketika login ditolak, tetapi password tidak dipulihkan. Fortify biasanya menempatkan kesalahan kredensial pada email sehingga error email ditampilkan. @csrf memasangkan request dengan sesi. autocomplete membantu browser mengisi jenis informasi yang benar.

Titik pemeriksaan: setelah seluruh view dan route dashboard pada bagian berikutnya dibuat, GET login harus menampilkan form dan POST login harus mengarah ke Fortify. Jangan mendaftarkan GET atau POST login tambahan di routes/web.php.

3D Membuat registrasi dan dashboard

Lengkapi antarmuka sebelum menguji autentikasi.

Petunjuk pengerjaan dan kode

Buat resources/views/auth/register.blade.php. Field name adalah nama akun pada tabel users; field nama dalam CRUD pendaftaran adalah data berbeda.

```
@extends('auth.layout')
@section('content')
<h1>Daftar akun</h1>
<form method="POST" action="{{ url('/register') }}">
    @csrf
    <label for="name">Nama akun</label>
    <input id="name" name="name" required maxlength="100"
        autocomplete="name" value="{{ old('name') }}">
    @error('name') <p class="error">{{ $message }}</p> @enderror
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required maxlength="255"
        autocomplete="username" value="{{ old('email') }}">
```

```
    @error('email') <p class="error">{{ $message }}</p> @enderror
    <label for="password">Password minimal 8 karakter</label>
    <input id="password" name="password" type="password" required
        minlength="8" autocomplete="new-password">
    @error('password') <p class="error">{{ $message }}</p> @enderror
    <label for="password_confirmation">Ulangi password</label>
    <input id="password_confirmation" name="password_confirmation"
        type="password" required autocomplete="new-password">
    <button>Daftar</button>
</form>
@endsection
```

password_confirmation harus ditulis persis karena aturan confirmed mencarinya. Email yang sudah terpakai harus ditolak. Kesalahan konfirmasi password ditampilkan pada kunci password. Form tidak menyertakan role atau hak admin karena fitur itu tidak dibuat dalam praktikum.

Pada proyek HMVC baru, tambahkan route dashboard ke routes/web.php dan pertahankan route welcome jika masih diperlukan. Jangan menambahkan require auth.php; endpoint autentikasi didaftarkan oleh Fortify.

```
use Illuminate\Support\Facades\Route;


Route::view('/dashboard', 'dashboard')
    ->middleware('auth')->name('dashboard');
```

Jika Route sudah diimpor di bagian atas file, cukup tambahkan deklarasi dashboard tanpa menduplikasi use. Buat resources/views/dashboard.blade.php berikut.

```
@extends('auth.layout')
@section('content')
<h1>Dashboard</h1>
<p>Selamat datang, {{ auth()->user()->name }}.</p>
<ul>
    <li><a href="{{ url('/pendaftaran') }}">CRUD biasa</a></li>
    <li><a href="{{ url('/pendaftaran/ajax') }}">CRUD AJAX</a></li>
</ul>
<p>Fitur pendaftaran tersedia setelah modulnya selesai dibuat.</p>
@endsection
```

Dashboard dilindungi auth sehingga auth user tersedia saat dirender. Link CRUD memakai URL; link boleh terlihat meskipun modul belum dibuat, tetapi jangan mengujinya sebelum bagian modul selesai. Logout sudah tersedia melalui layout auth.

Titik pemeriksaan: tidak ada route dashboard ganda. Dashboard mengembalikan view yang baru dibuat dan middleware auth terlihat pada route:list.

3E Menguji Fortify sebelum membuat CRUD

Uji jalur berhasil dan gagal secara terpisah.

Petunjuk pengerjaan dan kode

```
php artisan optimize:clear
php artisan route:list --path=login -v
php artisan route:list --path=register -v
php artisan route:list --path=logout -v
php artisan serve
```

Hasil yang dicari: GET login untuk form, POST login untuk proses, GET dan POST register jika registration aktif, serta POST logout. Lihat kolom Action untuk memastikan endpoint tersebut mengarah ke class Fortify. GET dan POST pada URI yang sama bukan duplikat karena metodenya berbeda.

1. Buka /register sebagai guest. Buat akun dengan email rekaan dan password minimal delapan karakter. Harapkan baris users baru, sesi login, serta dashboard. Periksa bahwa password pada database berupa hash tanpa menyalin atau menyebarkannya.

2. Logout melalui tombol. Buka /dashboard langsung. Harapkan pengalihan ke /login. Login dengan password benar dan pastikan dashboard kembali terbuka.

3. Logout lalu kirim password salah. Harapkan pesan login ditolak dan sesi tetap guest. Uji password_confirmation berbeda pada registrasi. Harapkan pesan validasi dan tidak ada akun baru.

4. Coba email yang sudah terdaftar. Harapkan penolakan keunikan email. Uji pembatas percobaan login secara terbatas; setelah ambang tercapai, tunggu periode pembatas sebelum menguji kredensial benar lagi.

5. Setelah modul CRUD selesai, buka pendaftaran tanpa login pada jendela privat. Uji juga request JSON tanpa sesi: harapkan 401. Request HTML biasa diarahkan ke login. Respons dapat berbeda sesuai Accept, meskipun keduanya memeriksa identitas yang sama.

Jika login berhasil tetapi diarahkan ke 404, periksa home dan route dashboard. Jika view login tidak ditemukan, periksa provider serta path. Jika ada bentrok route, jangan menambah controller baru; periksa apakah sistem autentikasi lama ikut mendaftarkan endpoint.

Selesai tahap ini berarti autentikasi siap dipakai oleh middleware auth pada modul. Fortify tidak membuat model Daftar atau CRUD pendaftaran; bagian berikutnya tetap diperlukan.

4 Menjalankan HMVC dengan sesi Fortify

Lanjutkan setelah bagian 3A sampai 3E selesai.

Petunjuk pengerjaan dan kode

```
php artisan serve
```

Buka alamat backend yang dicetak terminal, biasanya http://127.0.0.1:8000. Pada proyek HMVC, contoh view memakai CSS langsung dan JavaScript biasa sehingga tidak memerlukan npm run dev.

Daftar akun melalui /register, uji logout, lalu login ulang melalui /login. Setelah modul dibuat, buka /pendaftaran dan /pendaftaran/ajax. Semua endpoint CRUD harus berada dalam middleware auth.

Tujuan login mengikuti URL yang sebelumnya hendak dibuka jika tersedia; jika tidak, Fortify memakai home /dashboard. Gunakan host yang sama sepanjang praktik agar cookie sesi tetap konsisten.

Langkah 1 Memahami kebutuhan menjalankan HMVC

Pada proyek HMVC ini, cukup jalankan php artisan serve. Layout login dan CRUD memuat CSS langsung; AJAX berupa JavaScript di view. Tidak ada @vite pada layout contoh sehingga npm run dev tidak diperlukan untuk menampilkan latihan HMVC.

Fortify menyediakan endpoint, tetapi aplikasi menyediakan view dan dashboard. Jika /login gagal, cocokkan provider, konfigurasi views, serta file auth/login.blade.php sebelum menambah route baru.

Langkah 2 Membuktikan register login dan logout

Buka /register dan isi name, email, password, serta password_confirmation. Setelah berhasil, Anda masuk dan diarahkan ke dashboard. Kolom name pada tabel users berbeda dari kolom nama pada tabel daftars; jangan menyamakan keduanya tanpa mengubah aturan terkait.

Klik Logout yang menggunakan form POST dan token CSRF. Buka /dashboard lagi dan pastikan diarahkan ke /login. Login ulang dengan akun yang sama. Password salah harus menampilkan pesan tanpa membuat sesi login.

Langkah 3 Membedakan login dengan izin tindakan

Fortify mengurus proses autentikasi. Middleware auth pada route CRUD memeriksa sesi itu. Authentication membuktikan identitas, sedangkan authorization menentukan hak tindakan. Contoh tetap memakai data bersama untuk semua pengguna yang login.

Catat php artisan --version dan composer show laravel/fortify. Simpan lockfile hasil instalasi. Jangan memasang scaffolding autentikasi lain pada proyek ini karena dua sistem dapat mendaftarkan endpoint /login yang sama.

5 Memasang dan mengenali modul

Gunakan nwidart versi 12 untuk Laravel 12.

Petunjuk pengerjaan dan kode

```
composer require nwidart/laravel-modules:"~12.0"
php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider"
```

Pada composer.json utama, gabungkan konfigurasi berikut ke extra yang sudah ada. Jangan membuat dua properti extra dan jangan menghapus konfigurasi lain milik proyek.

```
"extra": {
```

```
    "laravel": { "dont-discover": [] },
    "merge-plugin": {
        "include": ["Modules/*/composer.json"]
    }
}
```

Izinkan wikimedia/composer-merge-plugin ketika Composer meminta. Jika sebelumnya ditolak, gunakan perintah konfigurasi di bawah. Pada paket v12 jangan menambahkan mapping umum Modules\ ke Modules/ dari tutorial lama; gunakan mapping composer.json tiap modul.

```
composer config allow-plugins.wikimedia/composer-merge-plugin true
php artisan module:make Pendaftaran
composer dump-autoload
php artisan module:list
```

```
Modules/Pendaftaran/
  app/Http/Controllers/PendaftaranController.php
  app/Models/Daftar.php       <- dibuat pada tahap berikutnya
  database/migrations/
  resources/views/
  routes/web.php
  composer.json
  module.json
```

Path memakai app/Models, tetapi namespace tetap Modules\Pendaftaran\Models karena mapping autoload modul. Pastikan status Pendaftaran aktif. Jika tidak aktif, jalankan php artisan module:enable Pendaftaran.

Langkah 1 Memasang paket modular

composer require menambahkan nwidart/laravel-modules ke dependensi PHP. Paket menyediakan generator, registrasi modul, route, dan pemuatan resource. vendor:publish menyalin konfigurasi atau stub yang disediakan paket ke proyek sehingga dapat disesuaikan. Jangan mengedit langsung berkas dalam vendor karena dapat tertimpa pembaruan.

Versi paket diselaraskan dengan versi Laravel acuan. Jika Composer menolak kompatibilitas, baca versi PHP dan Laravel; jangan memaksa pemasangan dengan mengabaikan persyaratan platform.

Langkah 2 Memahami autoload dan merge plugin

Autoload memungkinkan PHP menemukan class saat nama lengkapnya dipanggil. Setiap modul mempunyai composer.json dengan mapping namespace sendiri. Konfigurasi merge-plugin meminta Composer menggabungkan konfigurasi modul agar class di sana dikenali proyek utama.

Edit JSON dengan memperhatikan koma, tanda kutip ganda, dan pasangan kurung. Potongan extra harus digabung ke objek yang sudah ada. composer validate dapat dipakai untuk memeriksa struktur composer.json. allow-plugins mengizinkan plugin yang diperlukan berjalan; ia bukan nama modul aplikasi.

Langkah 3 Membuat modul dan memeriksa hasil

module:make Pendaftaran menghasilkan struktur awal. composer dump-autoload membangun ulang peta class setelah modul ada. module:list menunjukkan modul yang ditemukan beserta statusnya. Jika status nonaktif, enable mengaktifkannya; jika class tidak ditemukan, periksa mapping dan hasil Composer.

Buka composer.json milik Pendaftaran dan temukan mapping Modules\Pendaftaran\ ke app/. Itulah alasan nama namespace tidak mengandung app meskipun path berkas mengandung app. Periksa isi yang benar-benar dihasilkan generator sebelum menyalin path dari tutorial versi lain.

6 Membuat tabel dan model

Migration mengubah struktur database; model menggunakan tabelnya.

Petunjuk pengerjaan dan kode

```
php artisan module:make-model Daftar Pendaftaran --migration
```

Buka migration baru pada Modules/Pendaftaran/database/migrations. Pertahankan import, class, dan down yang dihasilkan. Ganti isi fungsi up seperti berikut.

```
public function up(): void
{
    Schema::create('daftars', function (Blueprint $table) {
        $table->id();
        $table->string('nama', 100);
        $table->string('asal_sekolah', 150);
        $table->string('prodi_tujuan', 100);
        $table->timestamps();
    });
}
```

Pastikan down memuat Schema::dropIfExists('daftars'). Jalankan migrasi modul. Jangan memakai migrate:fresh pada database berisi data penting karena perintah itu menghapus tabel.

```
php artisan module:migrate Pendaftaran
```

Ganti isi Modules/Pendaftaran/app/Models/Daftar.php.

```
<?php
namespace Modules\Pendaftaran\Models;
use Illuminate\Database\Eloquent\Model;


class Daftar extends Model
{
    protected $table = 'daftars';
    protected $fillable = [
        'nama', 'asal_sekolah', 'prodi_tujuan',
    ];
```

}

$fillable menentukan atribut yang boleh diisi massal melalui create atau update. Ia tidak menggantikan validasi. Titik pemeriksaan: tabel daftars memiliki id, tiga kolom isian, created_at, dan updated_at.

Langkah 1 Menghasilkan model dan migration

module:make-model menerima nama model lalu nama modul. Daftar adalah class model; Pendaftaran adalah modul pemiliknya. Opsi --migration meminta berkas perubahan skema sekaligus. Model dan migration bukan berkas yang sama dan memiliki peran berbeda.

Nama migration diawali timestamp agar urutan perubahan diketahui. Cari berkas yang baru dibuat, bukan mengedit sembarang migration proyek utama. Jika generator memberi nama tabel berbeda, samakan Schema::create dan down dengan daftars seperti contoh.

Langkah 2 Membaca setiap definisi kolom

Schema::create membuka pembuatan tabel. $table->id membuat primary key untuk membedakan setiap baris. string menyimpan teks dengan batas panjang; nama dibatasi 100, sekolah 150, dan prodi 100 karakter. timestamps menambahkan created_at dan updated_at. Ketiga isian tidak nullable sehingga wajib memiliki nilai.

Fungsi up menerapkan perubahan, sedangkan down mendefinisikan pembatalannya. Jika sebuah migration sudah dijalankan, menyunting file itu tidak otomatis mengubah tabel. Untuk latihan yang sudah memiliki data, buat migration perubahan baru. Hindari reset database sebagai jawaban pertama.

Langkah 3 Menjalankan dan memahami model

module:migrate Pendaftaran menjalankan migration modul tersebut. Periksa tabel melalui pengelola database dan cocokkan nama kolom. Catatan pada tabel migrations mencegah berkas yang sama dijalankan berulang.

Model extends Model mewarisi operasi Eloquent. protected $table menetapkan tabel secara eksplisit. protected $fillable membatasi atribut yang diizinkan pada create dan update massal. Kolom boleh ada di database tetapi tidak dapat diisi massal jika tidak dimasukkan ke fillable.

Contoh membaca satu perintah

Daftar::create menerima array hasil validasi, membentuk model, melakukan INSERT, dan mengembalikan model yang sudah memiliki ID. Daftar::latest('id')->get membaca kumpulan model dari ID terbesar. Nama variabel $row atau $rows tidak mengubah nama tabel.

7 Menentukan route yang dilindungi login

Ganti isi Modules/Pendaftaran/routes/web.php.

Petunjuk pengerjaan dan kode

```
<?php
use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;


Route::middleware('auth')->group(function () {
    Route::view('/pendaftaran/ajax', 'pendaftaran::ajax')
        ->name('pendaftaran.ajax');
    Route::resource('pendaftaran', PendaftaranController::class)
        ->except(['show']);
});
```

RouteServiceProvider bawaan modul memuat web.php dengan middleware web. Kelompok auth menambahkan syarat login. Route AJAX disiapkan sekarang, tetapi baru dibuka setelah view ajax dibuat.

```
php artisan route:list --path=pendaftaran -v
```

Cari GET /pendaftaran, GET /pendaftaran/create, POST /pendaftaran, GET /pendaftaran/{pendaftaran}/edit, PUT/PATCH /pendaftaran/{pendaftaran}, DELETE /pendaftaran/{pendaftaran}, dan GET /pendaftaran/ajax.

Route resource menyediakan nama pendaftaran.index, create, store, edit, update, dan destroy. Parameter {pendaftaran} diikat ke model melalui argumen Daftar $pendaftaran pada controller.

Kode di atas menggantikan route contoh yang dihasilkan generator. Jangan menyisakan nama route duplikat. Jika perubahan belum terbaca, jalankan php artisan optimize:clear.

Titik pemeriksaan: route:list menampilkan auth. Pengguna yang logout diarahkan ke /login ketika membuka /pendaftaran. Jika login belum dibuat, kembali ke bagian pemasangan dan pengujian Fortify.

Langkah 1 Mengganti route contoh generator

Route::resource menghasilkan pola CRUD yang konsisten. except show menghilangkan halaman detail yang tidak dibuat pada modul ini. Route::view untuk /pendaftaran/ajax hanya mengembalikan view AJAX, bukan endpoint data. Data JSON tetap diperoleh dari GET /pendaftaran dengan header Accept.

Route AJAX diletakkan lebih awal agar jalur statis mudah dikenali dan tidak tercampur dengan pola parameter saat route dikembangkan. Jangan membuka /pendaftaran/ajax sebelum file view tersedia; error view pada tahap itu berarti langkah tampilan belum dikerjakan.

Langkah 2 Membaca daftar route

route:list menampilkan metode, URI, nama, dan aksi. Opsi --path membatasi daftar agar fitur yang sedang diperiksa mudah terlihat. Opsi -v membantu menampilkan middleware. Cocokkan nama pendaftaran.store dengan POST dan pendaftaran.update dengan PUT/PATCH.

Nama route digunakan helper route untuk membuat URL. URI adalah alamat browser; nama route adalah identitas internal. Mengubah alamat tanpa mengubah nama dapat tetap mempertahankan pemanggilan helper di view.

Langkah 3 Memahami binding dan middleware

Ketika URL berisi /pendaftaran/7/edit, Laravel membaca parameter pendaftaran. Type hint Daftar pada argumen controller membuat Laravel mencari model dengan primary key tersebut. Nama parameter dan nama argumen harus cocok agar implicit binding bekerja seperti yang dimaksud.

Provider modul bawaan menangani kelompok web; auth ditambahkan pada kode latihan. Jangan mendaftarkan ulang web.php modul di routes/web.php utama karena dapat menghasilkan route ganda. Jika route tidak ditemukan, periksa status modul dan provider sebelum membuat pendaftaran manual.

Ringkasan pemetaan route

Metode	URI	Metode controller	Nama route
GET	/pendaftaran	index	pendaftaran.index
GET	/pendaftaran/create	create	pendaftaran.create
POST	/pendaftaran	store	pendaftaran.store
GET	/pendaftaran/{pendaftaran}/edit	edit	pendaftaran.edit
PUT atau PATCH	/pendaftaran/{pendaftaran}	update	pendaftaran.update
DELETE	/pendaftaran/{pendaftaran}	destroy	pendaftaran.destroy
GET	/pendaftaran/ajax	View langsung	pendaftaran.ajax

8 Controller untuk membaca dan menambah

Gabungkan kode bagian ini dan bagian berikutnya menjadi satu file.

Petunjuk pengerjaan dan kode

Ganti Modules/Pendaftaran/app/Http/Controllers/PendaftaranController.php. Jangan menutup class sebelum metode pada bagian berikutnya ditambahkan.

```
<?php
namespace Modules\Pendaftaran\Http\Controllers;


use App\Http\Controllers\Controller;
```

```
use Illuminate\Http\Request;
use Modules\Pendaftaran\Models\Daftar;


class PendaftaranController extends Controller
{
    public function index(Request $request)
    {
        $rows = Daftar::latest('id')->get();
        if ($request->wantsJson()) {
            return response()->json($rows);
        }
        return view('pendaftaran::index', compact('rows'));
    }


    public function create()
    {
        return view('pendaftaran::form', [
            'pendaftaran' => new Daftar(),
        ]);
    }


    public function store(Request $request)
    {
        $row = Daftar::create($this->validated($request));
        if ($request->wantsJson()) {
            return response()->json($row, 201);
        }
        return to_route('pendaftaran.index')
            ->with('status', 'Data berhasil ditambahkan.');
    }
```

index mengirim HTML untuk browser biasa atau JSON jika Accept meminta application/json. store hanya menyimpan data yang lolos validasi. Status 201 berarti data baru berhasil dibuat.

Langkah 1 Membaca pembuka controller

namespace menentukan alamat class di sistem autoload. use mengimpor class agar dapat ditulis singkat. Controller dasar berasal dari aplikasi, Request mewakili permintaan HTTP, dan Daftar adalah model milik modul. Nama file dan class harus sama.

public function adalah metode yang dapat dipanggil route. Kurung kurawal class membungkus seluruh metode. Kode di bagian berikutnya masih satu class, sehingga jangan menambah tag <?php kedua atau menutup class lebih awal.

Langkah 2 Menelusuri index dan create

latest id mengurutkan baris dari ID terbesar, lalu get mengeksekusi query dan menghasilkan koleksi. wantsJson memeriksa preferensi Accept. Jika JSON diminta, model dikonversi menjadi respons JSON; jika tidak, compact membuat array dengan kunci rows agar view menerima $rows.

create tidak menyimpan data. Ia menyiapkan new Daftar yang belum ada di database. Objek kosong ini memungkinkan form menggunakan properti exists untuk membedakan mode tambah dan edit. Menampilkan form berbeda dari menjalankan INSERT.

Langkah 3 Menelusuri store

Request dimasukkan ke validated terlebih dahulu. Jika validasi gagal, eksekusi create tidak dilanjutkan. Jika berhasil, hanya tiga field yang diizinkan oleh aturan yang diberikan kepada model. Respons JSON memakai status 201 untuk data baru.

Untuk form biasa, to_route membuat redirect menuju daftar. with status menyimpan pesan sementara pada session untuk request berikutnya. Pesan itu dibaca layout; tanpa kode session status di layout, penyimpanan tetap berhasil tetapi pesan tidak terlihat.

9 Controller untuk mengubah dan menghapus

Lanjutan file controller dari bagian sebelumnya.

Petunjuk pengerjaan dan kode

```
public function edit(Daftar $pendaftaran)
    {
        return view('pendaftaran::form', compact('pendaftaran'));
    }


    public function update(Request $request, Daftar $pendaftaran)
    {
        $pendaftaran->update($this->validated($request));
        if ($request->wantsJson()) {
            return response()->json($pendaftaran);
        }
        return to_route('pendaftaran.index')
            ->with('status', 'Data berhasil diperbarui.');
    }


    public function destroy(Request $request, Daftar $pendaftaran)
    {
        $pendaftaran->delete();
        if ($request->wantsJson()) {
            return response()->json(['message' => 'Data dihapus.']);
        }
        return to_route('pendaftaran.index')
            ->with('status', 'Data berhasil dihapus.');
    }


    private function validated(Request $request): array
    {
        return $request->validate([
            'nama' => ['required', 'string', 'max:100'],
            'asal_sekolah' => ['required', 'string', 'max:150'],
```

```
            'prodi_tujuan' => ['required', 'string', 'max:100'],
        ]);
    }
}
```

Route model binding memberikan 404 jika ID tidak ditemukan. Ini menghindari delete pada null. Validasi dipakai saat tambah maupun ubah. Input tidak disimpan menggunakan $request->all().

ID tidak perlu masuk $fillable. Untuk update, ID berasal dari URL dan diikat Laravel ke model yang sudah ada.

Langkah 1 Mengambil data untuk edit

edit menerima model yang sudah ditemukan oleh binding, lalu mengirimnya ke form. Pada tahap ini database belum diubah. Pengguna dapat meninggalkan form tanpa menyimpan dan data lama tetap utuh.

Jika ID tidak ada, binding menghasilkan 404. Ini lebih jelas daripada find yang dapat menghasilkan null lalu menyebabkan error saat propertinya dipanggil.

Langkah 2 Menyimpan perubahan dan menghapus

update dipanggil pada model yang sudah ada, sehingga baris dengan ID tersebut diubah, bukan ditambahkan. Aturan validasi yang sama dijalankan lagi. Data baru dari form tidak boleh dipercaya hanya karena pengguna sedang mengedit data lama.

destroy menghapus model lalu memilih JSON atau redirect sesuai request. Penghapusan pada contoh bersifat permanen; soft delete tidak diaktifkan. Konfirmasi berada di antarmuka, sedangkan endpoint tetap dilindungi auth di server.

Langkah 3 Memahami aturan validasi

required mewajibkan nilai; string mewajibkan jenis teks; max membatasi panjang teks. Batas ini dibuat sama dengan kolom database agar data terlalu panjang ditolak dengan pesan yang mudah dipahami, bukan dibiarkan menjadi error database.

private validated adalah metode bantu di controller, bukan Form Request terpisah. Return type array menyatakan hasilnya berupa array. Ia menghindari penulisan aturan yang sama dua kali. Pada proyek lebih besar, aturan dapat dipindahkan ke class Form Request.

Pemeriksaan terarah

Ubah asal sekolah sambil mempertahankan nama. Pastikan ID sebelum dan sesudah sama. Kemudian kirim nama kosong. Hasil benar adalah data lama tetap ada dan tidak berubah karena proses berhenti pada validasi.

10 Membuat layout Blade

Buat Modules/Pendaftaran/resources/views/layout.blade.php.

Petunjuk pengerjaan dan kode

Layout dibuat mandiri agar tidak bergantung pada @yield atau @stack dari layout aplikasi lain. Contoh tidak membutuhkan Bootstrap, jQuery, atau koneksi CDN.

```
<!doctype html>
<html lang="id">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>Pendaftaran</title>
    <style>
        body {font:16px Arial;max-width:960px;margin:32px auto;
              padding:0 16px;line-height:1.6}
        table {border-collapse:collapse;width:100%}
        th,td {border:1px solid #ccc;padding:8px;text-align:left}
        input {display:block;padding:8px;margin:4px 0 12px}
        button,a {margin-right:8px} .error {color:#b91c1c}
    </style>
</head>
<body>
    <nav><a href="/dashboard">Dashboard</a>
        <a href="{{ route('pendaftaran.index') }}">CRUD biasa</a>
        <form method="POST" action="{{ url('/logout') }}"
              style="display:inline">
            @csrf <button>Logout</button>
        </form>
    </nav>
    @if(session('status')) <p>{{ session('status') }}</p> @endif
    @yield('content')
    @stack('scripts')
</body>
</html>
```

@yield menempatkan isi section content dari view anak. @stack menempatkan JavaScript yang dikirim melalui @push. Jika pasangan ini tidak ada, isi atau script dapat tidak tampil walaupun view anak benar.

Meta csrf-token akan dibaca fetch. Untuk form HTML biasa, token disertakan menggunakan @csrf. Keduanya berkaitan dengan sesi aplikasi yang sama.

Nama pendaftaran::layout berarti view layout milik modul Pendaftaran. Tanda :: adalah namespace view modul, bukan nama folder literal.

Langkah 1 Membuat struktur dokumen HTML

doctype menyatakan dokumen HTML; lang membantu pembaca layar memahami bahasa; charset utf-8 menjaga karakter teks; viewport membantu tampilan pada layar kecil. title adalah nama tab browser, bukan judul tabel data.

CSS pada contoh hanya mengatur keterbacaan. Ia tidak menyimpan data dan tidak menjalankan validasi server. Tanpa CSS, alur CRUD tetap bekerja walau tampilan menjadi lebih sederhana.

Langkah 2 Menghubungkan layout dan view anak

Layout adalah kerangka bersama. View anak memakai extends, lalu menulis section content. Blade memasukkan section itu tepat pada yield content. Nama section harus sama; content dan contents merupakan dua nama berbeda.

Script AJAX dikirim dengan push scripts. Layout harus menyediakan stack scripts setelah elemen form dan tabel, sehingga ketika script membaca querySelector, elemen yang dicari sudah ada. Jika script dijalankan sebelum elemen ada, hasil pencarian dapat null.

Langkah 3 Memahami status dan token

session status menampilkan pesan flash dari controller satu kali setelah redirect. Meta csrf-token menyimpan token sesi yang nanti dibaca JavaScript. Token ini bukan password pengguna dan tidak menggantikan pemeriksaan login.

Helper route membentuk alamat CRUD biasa dari nama route. Link dashboard mengarah ke halaman dashboard yang dibuat pada bagian 3D. Jika dashboard tidak ada, periksa pemasangan auth dan route utama, bukan mengganti URL pendaftaran secara acak.

11 Membuat halaman daftar Blade

Buat Modules/Pendaftaran/resources/views/index.blade.php.

Petunjuk pengerjaan dan kode

```
@extends('pendaftaran::layout')
@section('content')
<h1>Daftar Pendaftaran</h1>
<a href="{{ route('pendaftaran.create') }}">Tambah data</a>
<table>
<thead><tr><th>Nama</th><th>Asal sekolah</th>
<th>Prodi tujuan</th><th>Aksi</th></tr></thead>
<tbody>
@forelse($rows as $row)
<tr>
    <td>{{ $row->nama }}</td><td>{{ $row->asal_sekolah }}</td>
    <td>{{ $row->prodi_tujuan }}</td>
    <td>
        <a href="{{ route('pendaftaran.edit', $row) }}">Edit</a>
```

```
        <form method="POST" style="display:inline"
              action="{{ route('pendaftaran.destroy', $row) }}"
              onsubmit="return confirm('Hapus data ini?')">
            @csrf @method('DELETE')
            <button>Hapus</button>
        </form>
    </td>
</tr>
@empty
<tr><td colspan="4">Belum ada data.</td></tr>
@endforelse
</tbody></table>
@endsection
```

Nama file adalah index.blade.php, bukan indeks.blade.php. $rows berasal dari controller index. @forelse mengulang data dan menyediakan kondisi kosong melalui @empty.

Form hapus menggunakan POST pada HTML, kemudian @method menambahkan penanda DELETE yang dikenali Laravel. @csrf menyertakan token. Konfirmasi di browser membantu mencegah salah klik.

Ekspresi {{ }} melakukan escaping teks. Jangan menggantinya dengan output HTML mentah untuk menampilkan input pengguna.

Langkah 1 Menggambar daftar dari server

Controller mengirim koleksi rows. forelse mengambil satu model setiap putaran dan menamakannya row. Satu putaran menghasilkan satu tr. Nilai nama, asal_sekolah, dan prodi_tujuan dimasukkan ke td. Nomor ID tidak wajib ditampilkan agar tetap dapat dipakai sebagai parameter route.

Saat koleksi kosong, bagian empty menghasilkan satu baris dengan colspan 4 karena tabel memiliki empat kolom. Ini membedakan tabel kosong yang sah dari script atau query yang gagal.

Langkah 2 Menghubungkan edit dan hapus

Link edit hanya membuka halaman form sehingga menggunakan GET. Helper route menerima model row dan mengambil identitasnya untuk URL. Form hapus melakukan tindakan yang mengubah data, sehingga tidak dibuat menjadi link GET.

onsubmit memanggil confirm. Nilai false membatalkan pengiriman; true melanjutkan. @method DELETE menghasilkan hidden field _method, sedangkan @csrf menghasilkan token _token. Laravel membaca keduanya ketika POST HTML diterima.

Langkah 3 Memeriksa keamanan keluaran

Blade {{ }} mengubah karakter khusus menjadi teks aman untuk konteks HTML. Input Rani akan ditampilkan sebagai teks. Jangan memakai {!! !!} untuk kolom yang diisi pengguna hanya agar tampilannya tampak berbeda.

Bukti selesai: ketika data ditambah lalu halaman daftar dibuka, jumlah tr data sesuai jumlah baris database. Jika tidak, periksa apakah controller mengirim variabel rows dengan nama yang sama.

12 Membuat satu form untuk tambah dan edit

Buat Modules/Pendaftaran/resources/views/form.blade.php.

Petunjuk pengerjaan dan kode

```
@extends('pendaftaran::layout')
@section('content')
<h1>{{ $pendaftaran->exists ? 'Edit' : 'Tambah' }} Pendaftaran</h1>
<form method="POST" action="{{ $pendaftaran->exists
    ? route('pendaftaran.update', $pendaftaran)
    : route('pendaftaran.store') }}">
    @csrf
    @if($pendaftaran->exists) @method('PUT') @endif
    @foreach(['nama','asal_sekolah','prodi_tujuan'] as $field)
        <label for="{{ $field }}">
            {{ ucwords(str_replace('_',' ',$field)) }}
        </label>
        <input id="{{ $field }}" name="{{ $field }}" required
            maxlength="{{ $field === 'asal_sekolah' ? 150 : 100 }}"
            value="{{ old($field, $pendaftaran->$field) }}">
        @error($field) <p class="error">{{ $message }}</p> @enderror
    @endforeach
    <button>Simpan</button>
    <a href="{{ route('pendaftaran.index') }}">Batal</a>
</form>
@endsection
```

Pada tambah, controller mengirim new Daftar sehingga exists bernilai false dan tujuan form adalah store. Pada edit, model sudah tersimpan sehingga exists bernilai true, tujuan form update, dan @method PUT ditambahkan.

old mengembalikan input sebelumnya ketika validasi gagal. Jika tidak ada input lama, nilai dari model digunakan. @error menampilkan pesan untuk kolom terkait.

required dan maxlength memeriksa input di browser. Aturan controller tetap wajib karena pemeriksaan browser dapat dilewati. Nama field pada HTML harus sama dengan aturan validasi dan nama kolom database.

Langkah 1 Memilih mode form

exists bernilai false untuk model baru dan true untuk model tersimpan. Operator kondisi ? : memilih judul serta tujuan form. Saat tambah, action menunjuk store; saat edit, action menunjuk update dan parameter model disertakan.

HTML tetap menuliskan method POST. Khusus edit, @method PUT memberi tahu Laravel bahwa ini operasi perubahan. Jika penanda tersebut hilang, request dapat tidak cocok dengan route atau salah menuju operasi tambah.

Langkah 2 Menjaga kesamaan nama

foreach menghasilkan tiga pasangan label dan input dari daftar field. name menentukan kunci data yang dikirim ke server; id menghubungkan label dan elemen browser. Keduanya sama dalam contoh agar mudah dipetakan, tetapi fungsinya berbeda.

str_replace mengubah garis bawah menjadi spasi dan ucwords memperbesar huruf awal kata untuk label. Transformasi label tidak mengubah name, sehingga asal_sekolah tetap cocok dengan kolom database.

Langkah 3 Menjaga input ketika gagal

old memprioritaskan input dari request yang gagal validasi. Jika tidak ada input lama, nilai properti model dipakai. Tanpa old, mahasiswa dapat mengira semua data hilang ketika server menolak satu kolom.

@error memeriksa pesan untuk field tertentu. Atribut required dan maxlength membantu pengguna sebelum submit, tetapi controller tetap menjadi pemeriksaan utama. Tombol Batal adalah link menuju daftar dan tidak memanggil store atau update.

13 Memeriksa CRUD biasa

Jangan lanjut ke AJAX sebelum pemeriksaan berikut berhasil.

Petunjuk pengerjaan dan kode

1. Login dan buka /pendaftaran. Ketika kosong, pesan Belum ada data muncul. Tambah Rani, SMA 1, Informatika. Setelah simpan, halaman daftar dimuat kembali dan satu baris terlihat.

2. Edit Rani menjadi Rani Putri. Jumlah baris tetap satu. Jika justru bertambah, periksa URL update, @method PUT, dan pemanggilan update pada controller.

3. Klik Hapus lalu batalkan. Baris tetap ada. Ulangi dan setujui. Baris hilang setelah halaman dimuat kembali.

4. Untuk menguji server, hapus atribut required melalui alat pengembang browser lalu kirim form kosong. Pesan per kolom muncul dan database tidak bertambah. Nilai input lain yang sudah benar tetap dipertahankan oleh old.

5. Buka /pendaftaran/999999/edit dengan ID yang tidak ada. Hasilnya 404. Logout lalu buka /pendaftaran. Hasilnya halaman login. Pastikan endpoint tambah, ubah, dan hapus juga berada dalam kelompok auth.

6. Tambah data lalu refresh browser. Data harus tetap ada di database. Amati Network: form biasa memicu pengiriman lalu navigasi dokumen halaman.

Catatan skala: get mengambil seluruh baris dan cocok untuk data praktikum kecil. Untuk data besar, gunakan pagination dan sesuaikan tampilan. Jangan menganggap pola ini cukup untuk semua kapasitas aplikasi.

Refleksi: tunjukkan berkas tempat input divalidasi, daftar kolom yang boleh disimpan, dan baris yang mengarahkan pengguna kembali ke daftar.

Cara mencatat pengujian yang dapat diulang

Untuk setiap kasus, catat kondisi awal, tindakan, hasil yang diharapkan, hasil aktual, dan status lulus atau perlu perbaikan. Contoh: awal satu baris Rani; tindakan ubah menjadi Rani Putri; harapan satu baris dengan ID sama; aktual dilihat di tabel dan database.

Jangan hanya mengandalkan screenshot halaman sukses. Bukti validasi gagal harus menunjukkan pesan dan bahwa jumlah data tidak bertambah. Bukti login harus diambil pada sesi yang benar-benar logout.

Menguji aturan browser dan server secara terpisah

Browser dapat menghentikan submit ketika required kosong. Untuk menguji Laravel, buka Inspect pada input dan hapus required sementara, lalu submit. Perubahan pada browser ini tidak mengubah kode sumber proyek. Refresh mengembalikan atribut awal.

Untuk batas panjang, kirim teks lebih panjang melalui alat uji HTTP yang menyertakan sesi dan token, atau ubah maxlength sementara pada browser. Harapkan pesan validasi, bukan SQLSTATE. Jangan menonaktifkan CSRF hanya untuk mempermudah pengujian.

Menentukan kapan boleh lanjut

Lanjut ke AJAX hanya jika tambah, baca, edit, hapus, validasi, 404, dan pengalihan login sudah lulus. AJAX menambah lapisan JavaScript; ia tidak memperbaiki model atau database yang sejak awal belum bekerja.

Contoh catatan hasil uji

Kondisi awal	Tindakan	Harapan	Bukti
Satu baris ID 7	Ubah nama	ID sama dan jumlah tetap satu	Daftar dan database
Nama wajib diisi	Kirim kosong	Ditolak dan tidak tersimpan	Pesan validasi
Belum login	Buka daftar	Diarahkan ke login	URL dan tampilan

14 Menyiapkan tampilan AJAX

Gabungkan kode bagian 14 sampai 17 dalam satu file ajax.blade.php.

Petunjuk pengerjaan dan kode

Buat Modules/Pendaftaran/resources/views/ajax.blade.php. AJAX memakai tabel dan controller yang sama agar perbedaannya mudah diamati. Tidak ada modal atau pustaka JavaScript tambahan.

```
@extends('pendaftaran::layout')
@section('content')
<h1>Pendaftaran dengan AJAX</h1>
<p>Tambah dan edit memakai form yang sama.</p>
<form id="form">
    <input type="hidden" id="record-id">
    @foreach(['nama','asal_sekolah','prodi_tujuan'] as $field)
        <label for="{{ $field }}">
            {{ ucwords(str_replace('_',' ',$field)) }}
        </label>
        <input id="{{ $field }}" name="{{ $field }}" required
            maxlength="{{ $field === 'asal_sekolah' ? 150 : 100 }}">
        <p class="error" id="{{ $field }}-error"></p>
    @endforeach
    <button id="save">Simpan</button>
    <button type="button" id="cancel">Batal edit</button>
</form>
<p id="message" role="status"></p>
<table>
    <thead><tr><th>Nama</th><th>Asal sekolah</th>
    <th>Prodi tujuan</th><th>Aksi</th></tr></thead>
    <tbody id="rows"></tbody>
</table>
@endsection
@push('scripts')
<script>
const base = @json(route('pendaftaran.index'));
const form = document.querySelector('#form');
const rows = document.querySelector('#rows');
const message = document.querySelector('#message');
const save = document.querySelector('#save');
const recordId = document.querySelector('#record-id');
const fields = ['nama', 'asal_sekolah', 'prodi_tujuan'];
let currentRows = [];
```

ID tersembunyi diisi saat edit dan dikosongkan saat tambah. URL dibuat Laravel sehingga tidak bergantung pada pengetikan alamat secara manual.

Langkah 1 Menyiapkan elemen yang akan diperbarui

Form yang sama dipakai tambah dan edit. record-id tidak memiliki name sehingga tidak ikut FormData; ID digunakan hanya untuk menentukan URL update. Ketika kosong, request adalah tambah. Saat terisi, request adalah ubah.

Elemen message menampilkan hasil umum dan memiliki role status. Masing-masing kolom memiliki elemen error sendiri. tbody rows menjadi tempat JavaScript menggambar baris yang diterima dari server.

Langkah 2 Menyimpan referensi elemen

querySelector mencari elemen berdasarkan selector CSS. #form berarti elemen dengan id form. Referensi disimpan agar tidak perlu mencari ulang untuk setiap operasi. Jika id pada HTML berubah, selector JavaScript juga harus diubah.

fields adalah daftar nama kolom yang dipakai berulang untuk membaca nilai, menggambar sel, dan menampilkan error. currentRows menyimpan salinan daftar terakhir di browser agar tombol edit dapat mengisi form. Salinan ini bukan pengganti database.

Langkah 3 Memahami alamat dasar

@json mengubah URL hasil route menjadi literal JavaScript yang valid. URL dasar adalah /pendaftaran. Untuk update dan delete, ID ditambahkan ke akhir. Jangan memakai /pendaftaran/ajax sebagai endpoint penyimpanan karena route itu hanya menampilkan halaman.

15 Mengirim request dan membaca error

Lanjutan script dalam ajax.blade.php.

Petunjuk pengerjaan dan kode

```
async function request(url, options = {}) {
    const response = await fetch(url, {
        ...options,
        credentials: 'same-origin',
        headers: {
            'Accept': 'application/json',
            'Content-Type': 'application/json',
            'X-CSRF-TOKEN': document.querySelector(
                'meta[name="csrf-token"]').content,
            ...(options.headers || {}),
        },
    });
    const json = await response.json().catch(() => ({}));
    if (!response.ok) {
        const error = new Error(json.message ||
            `Permintaan gagal (${response.status}).`);
        error.status = response.status;
        error.errors = json.errors || {};
        throw error;
    }
    return json;
}
function resetForm() {
    form.reset();
    recordId.value = '';
    save.textContent = 'Simpan';
    for (const field of fields) {
        document.getElementById(field + '-error').textContent = '';
    }
```

```
}
function showError(error) {
    message.textContent = error.status === 401
        ? 'Sesi berakhir. Silakan login kembali.'
        : error.status === 419
        ? 'Token kedaluwarsa. Muat ulang halaman.'
        : error.message;
    for (const field of fields) {
        document.getElementById(field + '-error').textContent =
            error.errors?.[field]?.[0] || '';
    }
}
```

fetch tidak otomatis melempar error untuk status 422 atau 500. Karena itu response.ok diperiksa. Accept meminta JSON; token CSRF melindungi request yang mengubah data.

Langkah 1 Membungkus fetch dengan fungsi request

async menyatakan fungsi menghasilkan Promise. await menunggu respons tanpa memblokir seluruh browser. options memuat metode dan body bila diperlukan. Spread ...options menyalin opsi, sedangkan header digabung agar nilai bawaan tetap tersedia.

credentials same-origin membuat cookie sesi disertakan pada asal yang sama. Accept application/json meminta respons JSON, Content-Type application/json menjelaskan body, dan X-CSRF-TOKEN membawa token untuk operasi yang mengubah data. GET tidak memakai body JSON pada contoh ini.

Langkah 2 Memeriksa HTTP dan format respons

response.json membaca body sebagai JSON. catch di sana menyediakan objek kosong bila body bukan JSON sehingga error HTTP masih dapat dijelaskan lewat status. response.ok hanya true untuk rentang 200 sampai 299; status 422 tetap harus dianggap gagal walau fetch sendiri tidak menolak Promise.

Objek Error diberi status dan errors agar pemanggil dapat membedakan sesi habis, validasi gagal, dan kegagalan umum. throw memindahkan alur ke catch pada pemanggil. Gangguan jaringan juga masuk catch, walau tidak selalu memiliki status HTTP.

Langkah 3 Menghapus kondisi lama dan menampilkan pesan

resetForm mengosongkan input, ID edit, label tombol, dan pesan per field. Jika ID tidak dihapus, pengguna dapat mengira sedang menambah padahal masih mengubah data lama.

showError memilih pesan umum lalu mengisi error setiap field. Optional chaining ?. mencegah pembacaan properti yang tidak ada; [0] memilih pesan pertama. Operator || memberi teks kosong jika field tidak memiliki error.

16 Menggambar tabel dan menyimpan

Lanjutan script dalam ajax.blade.php.

Petunjuk pengerjaan dan kode

```
async function loadData() {
    currentRows = await request(base);
    rows.replaceChildren();
    if (currentRows.length === 0) {
        const td = rows.insertRow().insertCell();
        td.colSpan = 4;
        td.textContent = 'Belum ada data.';
        return;
    }
    for (const item of currentRows) {
        const tr = rows.insertRow();
        for (const field of fields) {
            tr.insertCell().textContent = item[field];
        }
        const actions = tr.insertCell();
        for (const [action, label] of
             [['edit', 'Edit'], ['delete', 'Hapus']]) {
            const button = document.createElement('button');
            button.type = 'button';
            button.textContent = label;
            button.dataset.action = action;
            button.dataset.id = item.id;
            actions.append(button);
        }
    }
}
form.addEventListener('submit', async (event) => {
    event.preventDefault();
    save.disabled = true;
    message.textContent = '';
    for (const field of fields) {
        document.getElementById(field + '-error').textContent = '';
    }
    const id = recordId.value;
    const data = Object.fromEntries(new FormData(form));
    try {
        await request(id ? `${base}/${id}` : base, {
            method: id ? 'PUT' : 'POST', body: JSON.stringify(data),
        });
        resetForm();
        message.textContent = 'Data tersimpan.';
        await loadData();
    } catch (error) { showError(error); }
    finally { save.disabled = false; }
});
```

textContent menampilkan input sebagai teks. Tombol simpan dinonaktifkan selama request agar klik berulang tidak membuat penyimpanan ganda.

Langkah 1 Membaca daftar dari server

loadData mengirim GET ke base. currentRows diperbarui dengan array respons. replaceChildren menghapus tampilan baris lama agar pemanggilan ulang tidak menduplikasi daftar. Jika array kosong, satu baris keterangan dibuat.

Untuk setiap item, insertRow membuat tr dan insertCell membuat td. textContent mengisi teks, bukan markup. Tombol dibuat terpisah dengan dataset action serta id sehingga tindakan dan target dapat dibaca saat diklik.

Langkah 2 Mencegah pengiriman form biasa

Listener submit tetap mendukung tombol maupun Enter dari keyboard. preventDefault menghentikan navigasi HTML biasa agar fetch yang menangani pengiriman. save.disabled dinyalakan sebelum await untuk mengurangi pengiriman ganda.

FormData mengumpulkan input yang memiliki name. Object.fromEntries mengubah pasangan kunci dan nilai menjadi objek. JSON.stringify mengubah objek menjadi string JSON yang cocok dengan Content-Type. Laravel kemudian membaca field melalui Request.

Langkah 3 Memilih tambah atau update

Jika ID kosong, URL adalah base dan metode POST. Jika ID ada, URL menjadi base/ID dan metode PUT. Controller yang sama dipanggil sesuai route. reset dilakukan setelah penyimpanan berhasil, bukan sebelum request selesai, agar input tidak hilang ketika validasi gagal.

Setelah simpan, loadData mengambil keadaan database terbaru. catch menampilkan kegagalan; finally selalu mengaktifkan tombol lagi. Jika simpan berhasil tetapi pembacaan ulang gagal, database mungkin sudah berubah. Periksa daftar sebelum mengulangi tambah agar tidak membuat duplikat.

17 Edit hapus dan pemeriksaan AJAX

Akhiri ajax.blade.php dengan kode berikut.

Petunjuk pengerjaan dan kode

```
rows.addEventListener('click', async (event) => {
    const button = event.target.closest('button[data-action]');
    if (!button) return;
    const id = Number(button.dataset.id);
    if (button.dataset.action === 'edit') {
        const item = currentRows.find(row => row.id === id);
        if (!item) return;
        resetForm();
        recordId.value = id;
        for (const field of fields) {
            document.getElementById(field).value = item[field];
        }
        save.textContent = 'Perbarui';
```

```
        document.getElementById('nama').focus();
        return;
    }
    if (!confirm('Hapus data ini?')) return;
    button.disabled = true;
    try {
        await request(`${base}/${id}`, { method: 'DELETE' });
        if (Number(recordId.value) === id) resetForm();
        message.textContent = 'Data dihapus.';
        await loadData();
    } catch (error) { showError(error); }
    finally { button.disabled = false; }
});
document.querySelector('#cancel').addEventListener('click', resetForm);
loadData().catch(showError);
</script>
@endpush
```

Login lalu buka /pendaftaran/ajax. Uji tambah, edit, batal edit, dan hapus. Pada Network, harapkan POST 201, PUT 200, DELETE 200, dan GET 200. Tidak ada pemuatan dokumen halaman baru ketika operasi berhasil.

Uji input kosong setelah required dinonaktifkan: respons 422 dan pesan per kolom tampil. Uji teks Rani: tampil sebagai teks, bukan elemen HTML. Refresh browser untuk membuktikan data tersimpan.

Jika pengguna lain telah menghapus baris, update dapat menghasilkan 404. Muat ulang daftar dan pilih data yang masih ada. AJAX tidak menghilangkan kemungkinan konflik data.

Langkah 1 Menggunakan event delegation

Tombol baris dibuat setelah halaman dimuat. Listener dipasang pada tbody yang sudah ada, lalu closest menemukan tombol yang menjadi sumber klik. Teknik ini disebut event delegation dan tetap bekerja setelah baris digambar ulang.

dataset.id adalah string. Number mengubahnya menjadi angka agar perbandingan dengan ID JSON cocok. find mengambil item dari currentRows untuk mengisi form. Data edit pada contoh berasal dari daftar terakhir, bukan request GET edit baru.

Langkah 2 Menjalankan edit dan hapus

Pada edit, reset menghapus kondisi form lama; ID lalu diisi kembali dan nilai ketiga kolom dimasukkan. Tombol menjadi Perbarui. Menekan Batal edit mengosongkan ID sehingga submit berikutnya kembali menjadi POST tambah.

Pada hapus, confirm harus disetujui sebelum request DELETE. Tombol sementara dinonaktifkan. Jika data yang dihapus sedang diedit, form ikut direset agar tidak menyimpan ke ID yang sudah hilang. loadData kemudian mengambil daftar baru.

Langkah 3 Menyalakan pemuatan awal

loadData pada akhir script dijalankan segera saat halaman siap. catch showError diperlukan karena kegagalan pemuatan awal juga harus terlihat. Penutup script dan endpush menutup struktur Blade yang dibuka sebelumnya.

Di Network, pilih filter Fetch/XHR agar request data mudah dilihat. Bedakan request dokumen awal yang normal dari request setelah klik. Tanpa refresh penuh berarti tidak ada navigasi dokumen baru pada operasi itu, bukan berarti tidak ada request jaringan.

18 Menyiapkan React Starter Kit dengan Fortify

Gunakan snapshot resmi Laravel 12 yang sudah memuat Fortify dan Inertia 2.

Petunjuk pengerjaan dan kode

Jalur ini berbeda dari pemasangan manual pada HMVC: starter kit React sudah memiliki Fortify, action, konfigurasi, serta halaman auth. Jangan menjalankan installer autentikasi lain di dalamnya. Kerjakan dari folder induk, dan gunakan Git untuk memilih snapshot sumber yang sama.

```
git --version
git clone https://github.com/laravel/react-starter-kit.git praktikum-react
cd praktikum-react
git checkout b4712e343ea9b1a684999026f127c6e091fd7427
composer install
```

Pada proyek baru yang belum mempunyai .env, jalankan salin berikut di PowerShell. Setelah itu edit .env, buat database praktikum_react, dan isi DB_DATABASE=praktikum_react serta kredensial MySQL yang sesuai.

```
Copy-Item .env.example .env
php artisan key:generate
```

Buka config/fortify.php dan ubah hanya array features menjadi registration seperti berikut. Pertahankan konfigurasi dan provider lainnya.

```
'features' => [
    Features::registration(),
],
```

Pada routes/web.php, ubah kelompok dashboard dari middleware auth dan verified menjadi auth saja untuk tahap inti ini. Ini menunda verifikasi email secara eksplisit, bukan menganggap login sama dengan verifikasi email.

```
Route::middleware(['auth'])->group(function () {
    Route::get('dashboard', function () {
        return Inertia::render('dashboard');
    })->name('dashboard');
```

```
});


php artisan config:clear
php artisan migrate
npm ci
npm run build
composer show laravel/fortify
php artisan make:model Biodata -m
php artisan make:controller BiodataController
```

Jalankan artisan serve dan npm run dev pada dua terminal. Buka /register dan buat akun proyek kedua. Kode CRUD React pada bagian berikutnya diletakkan di resources/js/pages/Biodata, dengan pages huruf kecil.

Snapshot sumber diambil dari repositori resmi pada commit b4712e343ea9b1a684999026f127c6e091fd7427. Ia mempertahankan Laravel 12 untuk kesamaan kelas. Dosen perlu memeriksa pembaruan dependensi dan lockfile sebelum digunakan di luar latihan.

Langkah 1 Mengambil snapshot starter kit yang sesuai

Periksa git --version. git clone menyalin repositori resmi ke folder praktikum-react. git checkout memilih snapshot sumber Laravel 12 yang sudah mencantumkan Fortify dan Inertia 2. Commit dipilih secara eksplisit agar isi tutorial tidak berubah ketika cabang utama beralih ke Laravel mayor lain.

Snapshot ini merupakan acuan belajar, bukan anjuran memakai dependensi lama tanpa pemeliharaan. Composer dan npm tetap perlu diperiksa oleh dosen sebelum kelas. Jika tidak memakai Git, dosen dapat membagikan arsip snapshot yang sama dan lockfile yang sudah diverifikasi.

Langkah 2 Menyiapkan lingkungan proyek kedua

composer install memasang kebutuhan PHP berdasarkan lockfile bila tersedia. Copy-Item menyalin .env.example hanya pada proyek baru yang belum memiliki .env. Atur praktikum_react pada .env sebelum migrate. key:generate mengisi key proyek baru satu kali.

npm ci memasang versi dari package-lock.json. Jika lockfile tidak tersedia pada salinan yang Anda terima, gunakan npm install lalu simpan lockfile baru. Jangan menyalin .env dan akun dari database HMVC secara otomatis.

Langkah 3 Mengenali autentikasi React berbasis Fortify

Provider bawaan starter kit menghubungkan Fortify::loginView ke Inertia::render auth/login dan registerView ke auth/register. Tampilan React berada di resources/js/pages/auth. Fortify memproses POST /login dan POST /register; React menampilkan form dan pesan.

Jangan menjalankan fortify:install lagi pada starter kit ini karena konfigurasi dan provider sudah disediakan. Jangan menyalin provider Blade dari proyek pertama: hal itu akan membuat halaman autentikasi React diganti oleh view Blade.

Langkah 4 Mengatur fitur inti dan menguji aplikasi

Untuk tahap awal, features hanya memuat registration. Konfigurasi ini menunda reset password, verifikasi email, dan 2FA agar kelas fokus pada register/login/logout; fitur tersebut tetap dapat ditambahkan setelah alur dasar dipahami. Syarat verified pada dashboard disesuaikan menjadi auth untuk latihan ini.

Jalankan artisan serve dan npm run dev pada dua terminal. Gunakan satu proyek pada satu waktu atau port serta nama cookie sesi yang berbeda. Buat akun melalui /register lalu coba logout dan login. Baru sesudah lulus buat model Biodata.

19 Database dan model biodata

Nama kolom harus konsisten pada migration model dan form.

Petunjuk pengerjaan dan kode

Buka migration create_biodatas_table yang baru dibuat. Pertahankan import, class, dan down. Isi fungsi up sebagai berikut.

```
public function up(): void
{
    Schema::create('biodatas', function (Blueprint $table) {
        $table->id();
        $table->string('nama', 100);
        $table->string('tempat_lahir', 100);
        $table->date('tanggal_lahir');
        $table->timestamps();
    });
}
```

Ganti app/Models/Biodata.php lalu jalankan php artisan migrate.

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;


class Biodata extends Model
{
    protected $table = 'biodatas';
    protected $fillable = [
        'nama', 'tempat_lahir', 'tanggal_lahir',
    ];
}
```

tanggal_lahir menggunakan kolom DATE. Contoh model tidak menambahkan cast datetime sehingga nilai tanggal dikirim sebagai YYYY-MM-DD yang dapat digunakan input type date.

Nama class menggunakan PascalCase, seperti Biodata atau DataMahasiswa, bukan Sentence case. Nama metode umumnya camelCase. Nama tabel dan kolom pada contoh memakai huruf kecil serta garis bawah.

Titik pemeriksaan: database praktikum_react memiliki tabel biodatas. Jangan menjalankan migrasi pada proyek atau database HMVC secara tidak sengaja.

Langkah 1 Menentukan tipe kolom

nama dan tempat_lahir berupa teks, sedangkan tanggal_lahir berupa DATE. Memakai DATE membuat tanggal dapat diperiksa serta diurutkan sebagai tanggal, bukan teks bebas seperti 12 Januari. Input browser mengirim format YYYY-MM-DD.

Kolom id menjadi kunci yang dipakai route edit, update, dan delete. timestamps mencatat waktu pembuatan dan perubahan baris; keduanya berbeda dari tanggal lahir yang merupakan data domain.

Langkah 2 Menetapkan model

Biodata berada di app/Models karena latihan React ini menggunakan MVC biasa. Namespace App\Models harus cocok. protected table menghindari keraguan tentang pluralisasi biodata. fillable hanya memuat tiga isian, bukan seluruh properti model.

Contoh sengaja tidak memakai cast tanggal ke objek datetime. Jika Anda menambahkan cast, periksa serialisasi JSON dan ubah format sebelum memberikannya kepada input date. Input date tidak menerima semua bentuk timestamp secara langsung.

Langkah 3 Memeriksa migrasi

Jalankan migrate dari praktikum-react lalu lihat tabel pada praktikum_react. Jika artisan mengatakan Nothing to migrate tetapi tabel yang Anda lihat tidak ada, periksa koneksi aktif dan database yang sedang dibuka. Jangan langsung membuat tabel manual karena skema dan riwayat migration bisa berbeda.

20 Controller biodata bagian pertama

Gabungkan dengan bagian berikutnya dalam satu controller.

Petunjuk pengerjaan dan kode

Ganti app/Http/Controllers/BiodataController.php. Inertia::render menerima nama komponen relatif terhadap folder pages, tanpa ekstensi .tsx.

```
<?php
namespace App\Http\Controllers;


use App\Models\Biodata;
use Illuminate\Http\Request;
use Inertia\Inertia;


class BiodataController extends Controller
{
    public function index()
    {
```

```
        return Inertia::render('Biodata/Index', [
            'biodatas' => Biodata::latest('id')->get(),
        ]);
    }

    public function create()
    {
        return Inertia::render('Biodata/Form');
    }

    public function store(Request $request)
    {
        Biodata::create($this->validated($request));
        return to_route('biodata.index');
    }

    public function edit(Biodata $biodatum)
    {
        return Inertia::render('Biodata/Form', [
            'biodata' => $biodatum,
        ]);
    }
```

index mengirim array biodatas sebagai props. create menampilkan form kosong. edit mengirim satu biodata ke form yang sama. Satu komponen form dipakai ulang supaya penanganan input dan error tetap konsisten.

Jangan mengembalikan JSON biasa dari store pada alur useForm ini. Setelah berhasil, redirect menuju route daftar memungkinkan Inertia memuat props terbaru.

Langkah 1 Mengirim props ke React

Inertia::render tidak membaca Blade biasa. Argumen pertama adalah nama halaman React; argumen kedua adalah data props. Kunci biodatas pada index harus cocok dengan parameter yang dibaca komponen Index. Jika kuncinya rows tetapi komponen membaca biodatas, data menjadi undefined.

Nama komponen bersifat case sensitive pada lingkungan tertentu. Biodata/Index dan biodata/index dapat menjadi path berbeda. Gunakan kapitalisasi persis dari contoh pada Fortify.

Langkah 2 Membedakan halaman dan tindakan

index dan create menghasilkan halaman. store menjalankan tindakan penyimpanan lalu redirect. edit menghasilkan halaman form dengan props biodata. Jangan membuat store menampilkan form kembali saat penyimpanan sukses, karena daftar tidak diperbarui sesuai alur yang diharapkan.

Biodata::create hanya dipanggil setelah validated berhasil. Redirect ke daftar menyebabkan controller index membaca ulang database. Dengan demikian React menerima data baru dari sumber yang sudah tersimpan.

Langkah 3 Menggunakan form yang sama

Form tambah tidak menerima biodata sehingga nilai awal kosong. Form edit menerima biodata sehingga nilai awal berasal dari database. Prop opsional pada TypeScript menyatakan dua keadaan ini secara eksplisit.

21 Controller dan route biodata

Validasi tambah dan ubah menggunakan aturan yang sama.

Petunjuk pengerjaan dan kode

```
public function update(Request $request, Biodata $biodatum)
    {
        $biodatum->update($this->validated($request));
        return to_route('biodata.index');
    }


    public function destroy(Biodata $biodatum)
    {
        $biodatum->delete();
        return to_route('biodata.index');
    }


    private function validated(Request $request): array
    {
        return $request->validate([
            'nama' => ['required', 'string', 'max:100'],
            'tempat_lahir' => ['required', 'string', 'max:100'],
            'tanggal_lahir' => [
                'required', 'date', 'before_or_equal:today',
            ],
        ]);
    }
}
```

Tambahkan import dan route berikut ke routes/web.php. Pertahankan route home, dashboard, serta require settings.php. Route autentikasi tetap didaftarkan oleh Fortify.

```
use App\Http\Controllers\BiodataController;


Route::middleware('auth')->group(function () {
    Route::resource('biodata', BiodataController::class)
        ->except(['show'])
        ->parameters(['biodata' => 'biodatum']);
});
```

Parameter biodatum ditetapkan secara eksplisit agar cocok dengan argumen Biodata $biodatum pada controller. Nama props yang dikirim ke React tetap biodata. Kedua nama tersebut memiliki konteks berbeda.

```
php artisan route:list --path=biodata -v
```

Titik pemeriksaan: tersedia enam route CRUD tanpa show, parameter {biodatum}, dan middleware auth. Tanggal lahir di masa depan ditolak oleh aturan server.

Langkah 1 Menjaga konsistensi update dan delete

update memakai model hasil binding dan aturan validasi yang sama dengan store. destroy menghapus model tersebut. Kedua tindakan mengembalikan redirect ke daftar, bukan JSON manual, karena pemanggilnya adalah klien Inertia.

before_or_equal:today menolak tanggal lahir setelah hari ini menurut timezone aplikasi. Pada kelas lintas timezone, periksa pengaturan timezone Laravel. Aturan server ini memberi contoh validasi domain yang tidak cukup diperiksa oleh tipe data TypeScript.

Langkah 2 Menghindari ambiguitas parameter

resource biodata dapat memunculkan kebingungan bentuk tunggal kata. parameters menetapkan nama placeholder menjadi biodatum secara eksplisit. Argumen Biodata $biodatum pada tiga metode harus mengikuti nama tersebut. Nama route tetap biodata.edit, biodata.update, dan biodata.destroy.

Prop yang dikirim ke komponen bernama biodata karena itu nama yang mudah dibaca di frontend. Perubahan nama parameter URL tidak mengharuskan nama prop sama, asalkan penetapannya jelas di controller.

Langkah 3 Mempertahankan route bawaan starter kit

Tambahkan use BiodataController pada bagian import routes/web.php. Tambahkan kelompok route CRUD tanpa menghapus route home, dashboard, maupun require settings.php yang sudah ada. Pada snapshot ini endpoint autentikasi didaftarkan Fortify, bukan melalui routes/auth.php.

Tidak perlu membuat POST /login sendiri. Gunakan route:list untuk memastikan satu endpoint login per metode. Route GET /login dan POST /login memang sama-sama diperlukan dan bukan duplikat karena metodenya berbeda.

22 Halaman daftar React

Buat resources/js/pages/Biodata/Index.tsx.

Petunjuk pengerjaan dan kode

```
import { Link, router } from '@inertiajs/react';
```

```
export type Biodata = {
    id: number;
    nama: string;
    tempat_lahir: string;
    tanggal_lahir: string;
};
type Props = { biodatas: Biodata[] };

export default function Index({ biodatas }: Props) {
    return (
        <main style={{ padding: 24 }}>
            <h1>Daftar Biodata</h1>
            <Link href="/biodata/create">Tambah data</Link>
            <table>
                <thead><tr>
                    <th>Nama</th><th>Tempat lahir</th>
                    <th>Tanggal lahir</th><th>Aksi</th>
                </tr></thead>
                <tbody>
                    {biodatas.length === 0 && (
                        <tr><td colSpan={4}>Belum ada data.</td></tr>
                    )}
                    {biodatas.map((row) => (
                        <tr key={row.id}>
                            <td>{row.nama}</td>
                            <td>{row.tempat_lahir}</td>
                            <td>{row.tanggal_lahir}</td>
                            <td>
                                <Link href={`/biodata/${row.id}/edit`}>
                                    Edit
                                </Link>
                                <button onClick={() => {
                                    if (confirm('Hapus data ini?')) {
                                        router.delete(`/biodata/${row.id}`);
                                    }
                                }}>Hapus</button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </main>
    );
}
```

Tipe Biodata mendefinisikan bentuk satu baris. Props memastikan biodatas merupakan array. key memakai ID yang stabil. JSX menampilkan teks dengan escaping bawaan React.

Langkah 1 Mendefinisikan tipe data

export type Biodata mendefinisikan bentuk satu baris dan dapat diimpor form. id bertipe number; tiga kolom lain bertipe string. type Props menyatakan biodatas adalah array dari bentuk tersebut. TypeScript dapat menandai kesalahan seperti salah eja tempat_lahir sebelum aplikasi dibuka.

Informasi tipe tidak melakukan pemeriksaan runtime pada server. Laravel tetap memvalidasi input dan harus mengirim bentuk data yang sesuai. Hindari mengganti semuanya menjadi any hanya agar pesan kesalahan hilang.

Langkah 2 Menggambar daftar

Komponen menerima props lewat argumen fungsi. Kondisi length nol menampilkan pesan kosong. map mengubah setiap objek menjadi satu baris JSX. key menggunakan ID agar React dapat melacak identitas baris ketika daftar berubah.

Link dari Inertia membuat navigasi tetap melalui integrasi Inertia. a biasa melakukan navigasi dokumen penuh; keduanya dapat membuka alamat, tetapi perilaku navigasinya berbeda.

Langkah 3 Menghapus melalui Inertia

onClick menerima fungsi yang dijalankan ketika tombol ditekan. confirm memeriksa keputusan pengguna, lalu router.delete mengirim request hapus. Jangan memanggil router.delete langsung pada atribut tanpa fungsi pembungkus karena tindakan dapat terjadi saat komponen dirender.

Ekspresi row.nama ditampilkan sebagai teks oleh React. Tidak perlu mengubahnya menjadi HTML mentah. Gunakan data rekaan seperti Rani untuk membuktikan bahwa keluaran tetap teks.

23 Form React untuk tambah dan edit

Buat resources/js/pages/Biodata/Form.tsx.

Petunjuk pengerjaan dan kode

```
import type { FormEvent } from 'react';
import { Link, useForm } from '@inertiajs/react';
import type { Biodata } from './Index';


type Data = {
    nama: string;
    tempat_lahir: string;
    tanggal_lahir: string;
};
export default function Form({ biodata }: { biodata?: Biodata }) {
    const { data, setData, post, put, processing, errors } = useForm<Data>({
        nama: biodata?.nama ?? '',
        tempat_lahir: biodata?.tempat_lahir ?? '',
        tanggal_lahir: biodata?.tanggal_lahir ?? '',
    });
    const labels: Record<keyof Data, string> = {
        nama: 'Nama', tempat_lahir: 'Tempat lahir',
        tanggal_lahir: 'Tanggal lahir',
    };
    function submit(event: FormEvent<HTMLFormElement>) {
        event.preventDefault();
        if (biodata) put(`/biodata/${biodata.id}`);
```

```
        else post('/biodata');
    }
    return (
        <main style={{ padding: 24 }}>
            <h1>{biodata ? 'Edit' : 'Tambah'} Biodata</h1>
            <form onSubmit={submit}>
                {(Object.keys(labels) as (keyof Data)[]).map((key) => (
                    <div key={key}>
                        <label htmlFor={key}>{labels[key]}</label>
                        <input id={key} required value={data[key]}
                            type={key === 'tanggal_lahir' ? 'date' : 'text'}
                            maxLength={key === 'tanggal_lahir' ? undefined : 100}
                            onChange={(e) => setData(key, e.target.value)} />
                        {errors[key] && <p role="alert">{errors[key]}</p>}
                    </div>
                ))}
                <button disabled={processing}>
                    {processing ? 'Menyimpan...' : 'Simpan'}
                </button>
                <Link href="/biodata">Batal</Link>
            </form>
        </main>
    );
}
```

useForm mengelola error dari alur Inertia. processing mencegah klik simpan berulang. Tipe event dan props menjaga pemeriksaan TypeScript tetap bermakna.

Langkah 1 Membentuk keadaan awal form

FormEvent diimpor sebagai tipe karena tidak dibutuhkan saat runtime. Data membatasi form pada tiga kolom. Prop biodata opsional: tanda ? berarti boleh tidak diberikan. Optional chaining ?. membaca nilai jika objek ada; ?? memakai string kosong jika nilainya null atau undefined.

useForm mengembalikan data, setData, fungsi kirim, status processing, dan errors. data adalah keadaan form saat ini. setData mengubah keadaan itu. post dan put mengirim melalui Inertia. errors berisi pesan validasi per nama kolom.

Langkah 2 Membuat input terkendali

Input memiliki value dari data dan onChange yang memanggil setData. Keduanya harus dipasangkan; value tanpa perubahan state membuat input tampak tidak dapat diketik. e.target.value membaca nilai terbaru dari elemen.

Record menghubungkan setiap kunci Data dengan label. Object.keys menghasilkan daftar kunci; as (keyof Data)[] menyatakan bahwa kunci label cocok dengan field form. map membuat tiga input tanpa menyalin struktur yang sama tiga kali.

Langkah 3 Mengirim dan menampilkan hasil

submit bertipe FormEvent<HTMLFormElement> agar event form dikenali TypeScript. preventDefault menghentikan submit HTML biasa. Jika biodata ada, put mengubah ID tersebut; jika tidak, post membuat data baru.

processing menonaktifkan tombol saat request berjalan dan mengganti teks menjadi Menyimpan. errors[key] ditampilkan dekat input terkait. Pada validasi gagal, pengguna tetap dapat memperbaiki nilai. Link Batal kembali ke daftar dan tidak mengirim data.

24 Memeriksa React dan autentikasi Fortify

Buktikan build tipe sesi dan proteksi CRUD.

Petunjuk pengerjaan dan kode

```
npm run build
npm run types
composer show laravel/fortify
php artisan route:list --path=login -v
```

Periksa resources/js/app.tsx: resolver menggunakan ./pages/${name}.tsx. Karena itu Biodata/Index dan Biodata/Form ditempatkan pada pages/Biodata, dengan nama file dan folder persis seperti nama render.

FortifyServiceProvider bawaan starter kit menghubungkan halaman auth React melalui Inertia. Pertahankan provider tersebut. Jangan menggantinya dengan provider Blade dari proyek HMVC.

Login melalui /login, tambah biodata, ubah satu kolom, dan hapus dengan konfirmasi. Refresh membuktikan penyimpanan. Uji tanggal lahir masa depan, ID tidak ada, serta akses saat logout.

Gunakan menu logout bawaan starter kit. Endpointnya POST /logout. Jika menambahkan tombol sendiri pada halaman latihan, panggil router.post('/logout') melalui handler klik, bukan link GET.

Middleware auth memeriksa sesi yang dibuat Fortify. Inertia mengatur navigasi dan props, sedangkan Fortify mengatur proses autentikasi. Perannya berbeda walaupun sama-sama digunakan dalam satu aplikasi.

Langkah 1 Memeriksa build dan tipe

npm run build memeriksa bundling aset; npm run types menjalankan TypeScript tanpa menghasilkan berkas. Jika gagal, perbaiki pesan pertama yang menunjuk berkas Anda. Jangan mengganti semua tipe menjadi any agar error hilang.

Periksa pula composer show laravel/fortify dan php artisan route:list --path=login. Dengan cara itu mahasiswa membuktikan bahwa login benar-benar berasal dari Fortify, bukan sekadar tampilan bertuliskan login.

Langkah 2 Memeriksa alur CRUD setelah login

Tambah data lalu refresh untuk memastikan persistensi. Edit satu kolom dan periksa ID serta jumlah baris tetap. Uji tanggal lahir masa depan, batalkan hapus, setujui hapus, ID tidak ada, dan akses sebelum login.

Request Inertia membawa protokol halaman dan props; ia berbeda dari fetch JSON biasa. Validasi gagal dibaca melalui errors pada useForm. Sesi login tetap sesi Laravel web yang dibentuk Fortify.

Langkah 3 Mengembangkan autentikasi sesudah dasar lulus

Reset password memerlukan view terkait serta pengiriman email atau driver log untuk latihan. Verifikasi email memerlukan kontrak model yang sesuai, alur notifikasi, dan middleware verified pada route yang memang dibatasi. 2FA memerlukan trait, kolom, challenge, serta pemulihan akun. Aktifkan satu fitur pada satu waktu dan uji alurnya.

Pada starter kit, pertahankan action dan provider bawaan untuk fitur yang Anda pakai. Konfigurasi Blade di proyek HMVC adalah contoh terpisah, bukan berkas yang ditempel ke proyek React.

25 Menemukan penyebab error

Baca bukti sebelum mengubah kode.

Petunjuk pengerjaan dan kode

Class not found: periksa namespace, use, lokasi file, composer.json modul, serta izin merge-plugin. Jalankan composer dump-autoload setelah konfigurasi diperbaiki.

View not found: cocokkan pendaftaran::index dengan resources/views/index.blade.php milik modul. indeks adalah nama berbeda. Pastikan modul aktif dan provider hasil generator tidak terhapus.

404: bedakan route tidak ada dengan ID tidak ditemukan. Periksa route:list dan data. 405: metode HTTP tidak cocok; periksa @method atau method fetch.

419: token atau sesi tidak cocok. Refresh, login ulang, gunakan host yang sama, dan periksa header token. 401 pada AJAX: sesi login tidak ada atau berakhir. Redirect ke login pada request HTML merupakan perilaku normal.

422: input gagal validasi. Baca errors pada Response atau Preview di Network. Pada Inertia, baca errors dari useForm karena alurnya memakai redirect dan session error, bukan penanganan fetch 422 langsung.

SQLSTATE connection refused: periksa layanan database, host, dan port. Access denied: periksa username/password. Unknown database: database belum dibuat. Table not found: migrasi belum berjalan pada koneksi aktif.

Vite manifest not found atau aset gagal: jalankan npm install lalu npm run dev selama pengembangan, atau npm run build untuk menghasilkan aset build. Baca pesan kebutuhan Node pada terminal.

Page not found React: cocokkan nama render, Pages/pages, .tsx, dan resolver. Error implicit any atau unknown: berikan tipe props dan event; jangan menonaktifkan pemeriksaan TypeScript.

500: baca storage/logs/laravel.log dan terminal. Saat bertanya, catat langkah pemicu, URL, metode, status, serta pesan inti. Jangan membagikan password .env.

Urutan diagnosis: server hidup -> route benar -> login valid -> periksa Network -> baca respons -> baca log -> perbaiki bagian terkait -> ulangi pengujian yang sama.

Langkah 1 Mengumpulkan bukti

Buka alat pengembang browser melalui menu Inspect, lalu tab Network. Aktifkan pencatatan, ulangi satu tindakan, dan pilih request yang baru. Catat method, URL, status, payload, dan response. Console dipakai untuk error JavaScript, sedangkan log Laravel dipakai untuk error server.

Bedakan tiga lapisan: browser belum mengirim request, request sudah terkirim tetapi ditolak server, atau server sukses tetapi tampilan gagal diperbarui. Perbaikannya berbeda untuk setiap lapisan.

Langkah 2 Memperbaiki penyebab yang terbukti

Untuk error namespace, cocokkan path dengan mapping lalu bangun ulang autoload. Untuk error SQL, cocokkan koneksi dan skema. Untuk error JavaScript, periksa elemen yang dicari dan nilai respons. Menghapus cache tidak memperbaiki typo field atau aturan bisnis yang salah.

Gunakan optimize:clear ketika memang ada kemungkinan konfigurasi atau route tersimpan. Jangan menjadikannya ritual pengganti membaca pesan. Setelah memperbaiki satu penyebab, ulangi kasus yang sama sebelum mencoba fitur lain.

Format laporan bantuan

Tulis: langkah yang dilakukan; hasil yang diharapkan; hasil aktual; metode dan URL; status HTTP; pesan utama; potongan kode terkait; serta perbaikan yang sudah dicoba. Hilangkan password, token sesi, dan data pribadi. Laporan seperti ini membantu dosen menelusuri masalah tanpa menebak seluruh proyek.

26 Tugas Modul 9 dan rubrik

Cakupan dan bukti keberhasilan dibuat eksplisit.

Petunjuk pengerjaan dan kode

Bagian A: demonstrasikan CRUD biasa dalam modul HMVC setelah login. Bagian B: demonstrasikan CRUD AJAX dengan entitas yang sama setelah login. Bagian C: CRUD biodata React menjadi latihan pembanding; dosen dapat menetapkannya sebagai komponen nilai tambahan atau pengganti.

Dalam rancangan tugas ini, minimal lima item berarti lima baris data per entitas yang diuji, bukan lima kolom atau lima modul. Dosen dapat mengubah definisi ini sebelum membagikan tugas.

Gunakan data rekaan: Rani / SMA 1 / Informatika; Bima / SMA 2 / Sistem Informasi; Siti / SMK 1 / Informatika; Deni / SMA 3 / Manajemen; Lala / SMK 2 / Akuntansi.

Kumpulkan kode sumber, composer.lock, package-lock.json, migration, README berisi versi dan cara menjalankan, serta bukti pengujian. Jangan menyertakan vendor, node_modules, atau .env berisi kredensial. Sediakan .env.example yang aman.

Bukti minimum: tambah lima data; ubah satu tanpa menambah jumlah baris; batalkan lalu setujui hapus; validasi gagal; ID tidak ada; akses sebelum login; perbedaan Network pada CRUD biasa dan AJAX.

Rubrik 100 poin: struktur modul dan route 15; CRUD benar 25; validasi dan kasus gagal 15; login pada semua endpoint 15; AJAX dan konsistensi tampilan 15; bukti uji dan README 10; penjelasan alur data 5. Jika React dinilai, tetapkan bobot pengganti sebelum tugas dimulai.

Refleksi: mengapa $fillable belum cukup? Apa beda 404 dan 422? Mengapa input tidak dimasukkan melalui innerHTML? Apa beda auth dan menyembunyikan tombol? Mengapa React tidak otomatis berarti HMVC?

Urutan pengerjaan tugas

Pilih satu entitas lalu tetapkan nama modul, model, tabel, kolom, dan route sebelum menulis kode. Buat diagram alur kecil, selesaikan CRUD biasa, lalu pakai backend yang sama untuk AJAX. Jika mengganti tema kasus, ubah semua nama secara konsisten, bukan hanya judul halaman.

Isi lima baris data rekaan yang berbeda. Jumlah minimum dihitung pada keadaan pengujian yang disepakati. Bila demonstrasi hapus membuat jumlah di bawah lima, tambahkan kembali data rekaan sebelum pengumpulan akhir.

Bukti yang harus dapat dinilai

README harus memungkinkan dosen memasang dependensi, membuat .env, menghubungkan database, menjalankan migration, dan membuka alamat fitur. Sertakan versi acuan dan cara membuat akun latihan. Jangan hanya menulis jalankan aplikasi tanpa urutan.

Buat catatan pengujian dengan hasil aktual, bukan sekadar tanda centang sebelum diuji. Screenshot tidak harus menampilkan password atau seluruh isi .env. Video singkat dapat memperlihatkan perbedaan refresh penuh dan fetch, jika diizinkan dosen.

Pertanyaan saat demonstrasi

Dosen dapat memilih satu field dan meminta mahasiswa menelusurinya dari name input sampai kolom database. Mahasiswa juga diminta menjelaskan apa yang terjadi jika ID tidak ada, token kedaluwarsa, atau validasi gagal. Kemampuan menjelaskan menjadi bukti bahwa kode dipahami.

27 Panduan dosen dan referensi

Ajarkan satu alur kecil sebelum menambah kerumitan.

Petunjuk pengerjaan dan kode

Pertemuan 1: konsep dan persiapan 30 menit, database/login 30 menit, modul dan CRUD biasa 75 menit, pemeriksaan 15 menit. Pertemuan 2: penelusuran request 20 menit, AJAX 80 menit, uji gagal 30 menit, diskusi 20 menit. Pertemuan 3: React/Inertia 25 menit, CRUD 90 menit, uji dan refleksi 35 menit.

Mulai dari satu data. Minta mahasiswa menunjuk route, controller, model, tabel, dan view yang terlibat. Setelah tambah berhasil, lanjut edit dan hapus. Pada AJAX, tampilkan Network berdampingan dengan editor dan demonstrasikan kesalahan validasi secara sengaja.

Kunci refleksi: $fillable membatasi atribut mass assignment, validasi memeriksa isi; 404 berarti sumber daya tidak ditemukan, 422 berarti input ditolak; textContent mencegah input dianggap markup; auth memeriksa request di server; React mengatur antarmuka, modularitas mengatur organisasi backend.

Sebelum kelas, jalankan seluruh alur pada komputer acuan dan bagikan lockfile yang telah diperiksa. Ekstensi PHP, layanan database, dan dependensi lokal perlu dikonfirmasi lewat langkah instalasi dan pengujian.

Sumber awal: Modul 9 - HMVC dan Laravel Starterkit, PDF bahan ajar 19 halaman yang diberikan dosen. Referensi resmi berikut menjadi pendamping untuk struktur, kompatibilitas, dan perilaku framework.

Laravel Modules v12 instalasi dan autoload: https://laravelmodules.com/docs/12/getting-started/installation-and-setup

Laravel Modules v12 struktur: https://laravelmodules.com/docs/12/basic-usage/creating-a-module

Starter kit Laravel 12 modern: https://laravel.com/docs/12.x/starter-kits

Inertia v2 form: https://inertiajs.com/docs/v2/the-basics/forms

Fortify Laravel 12: https://laravel.com/docs/12.x/fortify

Snapshot resmi React Starter Kit Laravel 12: https://github.com/laravel/react-starter-kit/tree/b4712e343ea9b1a684999026f127c6e091fd7427

Intervensi saat mahasiswa kebingungan

Jika mahasiswa tidak tahu berkas yang harus dibuka, minta mereka menyebut tindakan pengguna dan metode HTTP lebih dahulu. Dari route, ikuti metode controller, model, lalu view. Urutan ini lebih membantu daripada langsung mencari seluruh kata yang mirip di proyek.

Jika mahasiswa hanya menyalin, minta prediksi sebelum tombol ditekan: berapa jumlah baris setelah edit, apakah ID berubah, respons apa yang diterima, dan bagian tampilan mana yang diperbarui. Setelah itu bandingkan prediksi dengan hasil nyata.

Menjaga lingkungan kelas

Dosen sebaiknya menyiapkan satu proyek acuan yang sudah melalui instalasi dan pengujian, kemudian membagikan lockfile serta daftar versi. Koneksi database dan ekstensi PHP perlu diperiksa pada komputer kelas karena dua komputer dapat memiliki PHP terminal yang berbeda.

Pisahkan penilaian konsep, keberhasilan fungsi, dan mutu penjelasan. Mahasiswa yang dapat menjelaskan penyebab error tetapi belum menyelesaikan satu detail membutuhkan bantuan berbeda dari mahasiswa yang memiliki aplikasi berjalan namun tidak mengenali route yang dipakai.

Lampiran A Glosarium istilah

Artisan

Alat perintah Laravel untuk menghasilkan berkas, menjalankan migration, memeriksa route, dan menjalankan pekerjaan aplikasi.

Autoload

Mekanisme menemukan class dari nama namespace berdasarkan mapping Composer.

Blade

Mesin template Laravel yang menggabungkan HTML dengan data dan directive sebelum dikirim ke browser.

Composer

Pengelola dependensi PHP. composer.json menyatakan kebutuhan; composer.lock menyimpan versi hasil resolusi.

Controller

Class yang mengatur pemrosesan request dan pemilihan respons.

CSRF

Perlindungan request berbasis sesi terhadap pengiriman tindakan dari sumber yang tidak semestinya. Token tidak menggantikan login.

DOM

Representasi elemen halaman di browser yang dapat dibaca dan diubah JavaScript.

Eloquent

Lapisan model Laravel yang menyediakan operasi database melalui objek dan query builder.

Endpoint

Tujuan request yang ditentukan oleh metode HTTP dan URL.

JSON

Format pertukaran data berbasis teks, misalnya objek dan array; berbeda dari HTML tampilan.

Migration

Berkas perubahan skema database yang dijalankan dan dicatat urutannya.

Middleware

Pemeriksaan atau pemrosesan di sekitar request sebelum atau setelah aksi utama.

Props

Data masukan komponen React, pada latihan ini dikirim dari controller melalui Inertia.

Route model binding

Pengambilan model berdasarkan parameter route sebelum metode controller memakainya.

Session

Penyimpanan keadaan antar request, termasuk identitas login dan flash message.

State

Keadaan data antarmuka yang dapat berubah, misalnya nilai form React.

Validation

Pemeriksaan input berdasarkan aturan sebelum input digunakan atau disimpan.

Vite

Alat pemrosesan aset frontend selama pengembangan dan saat menghasilkan build.

Fortify

Backend autentikasi Laravel untuk proses seperti login, logout, dan registrasi. Tampilan dapat berupa Blade buatan sendiri atau React dari starter kit.

Lampiran B Menelusuri satu data dari awal sampai akhir

Kasus tambah

Pengguna mengisi nama Rani, asal_sekolah SMA 1, dan prodi_tujuan Informatika. Nama field menjadi kunci request. Route POST /pendaftaran memilih store. auth memeriksa sesi. validated memeriksa isi. create menjalankan INSERT. Database memberi ID, misalnya 7. Controller mengirim redirect atau JSON sesuai pemanggil. View daftar atau loadData kemudian membaca hasil yang tersimpan.

Kasus edit

Pengguna memilih ID 7 dan mengganti prodi_tujuan menjadi Sistem Informasi. Form biasa membuka GET edit dahulu; AJAX mengambil nilai dari currentRows. Saat simpan, URL menyertakan /7 dan metode PUT. Binding menemukan ID 7. Validasi memeriksa ulang tiga field. update mengubah baris yang sama. ID tetap 7 dan jumlah baris tetap.

Kasus validasi gagal

Pengguna mengosongkan nama dan melewati required browser. Request tetap dapat mencapai Laravel. validated menolak sebelum create atau update. Pada Blade, input dan pesan dikembalikan lewat session. Pada fetch, respons JSON berisi errors dengan status 422. Pada Inertia, useForm menerima error melalui integrasi redirect. Database tidak berubah.

Kasus data sudah hilang

Pengguna mencoba mengubah ID 7 setelah baris itu dihapus. Binding tidak menemukan model sehingga respons 404 muncul. Jangan membuat baris pengganti secara diam-diam. Muat ulang daftar dan beri tahu pengguna bahwa data yang dipilih sudah tidak tersedia.

Lampiran C Contoh README praktikum

Identitas proyek

Tuliskan nama aplikasi, tujuan fitur, versi PHP/Laravel/Node, dan jenis database. Sebutkan bahwa data latihan adalah data rekaan.

Pemasangan

Salin proyek ke folder kerja; jalankan composer install dan npm ci jika package-lock.json tersedia; salin .env.example menjadi .env hanya jika belum ada; atur database; jalankan php artisan key:generate untuk proyek baru yang belum memiliki key; lalu jalankan php artisan migrate. Untuk proyek modular, jalankan composer dump-autoload setelah dependensi dan modul tersedia, lalu verifikasi module:list.

Menjalankan

Pada proyek HMVC, jalankan php artisan serve. Pada proyek React, jalankan php artisan serve di terminal pertama dan npm run dev di terminal kedua. Buka alamat backend yang dicetak terminal, buat akun melalui /register, lalu login. Sebutkan URL fitur yang tersedia sesuai proyek.

Pengujian dan batas fitur

Cantumkan hasil uji tambah, edit, hapus, validasi, 404, dan akses tanpa login. Nyatakan bahwa seluruh pengguna login berbagi data contoh dan belum ada pembatasan kepemilikan. Tuliskan masalah yang benar-benar masih ada jika ada.

Autentikasi Fortify

Pada HMVC, periksa provider, action CreateNewUser, config/fortify.php, dan view auth. Pada React, pertahankan provider dan halaman auth starter kit. Pastikan tidak ada endpoint login ganda dari sistem lain.

Lampiran D Lembar pengujian

Isi hasil aktual setelah benar-benar mencoba. Gunakan salinan lembar untuk proyek HMVC biasa, AJAX, dan React. Jika satu kasus gagal, catat langkah reproduksi dan perbaikannya.

Nama mahasiswa _____ Proyek _____

Tanggal _____ Versi Laravel _____

No	Kasus	Hasil yang diharapkan	Aktual dan status
1	Baca saat kosong	Pesan kosong yang jelas	
2	Tambah data valid	Baris tersimpan dan terlihat setelah refresh	
3	Ubah satu field	ID sama dan jumlah baris tetap	
4	Batalkan edit	Tidak ada perubahan database	
5	Batalkan hapus	Baris tetap ada	
6	Setujui hapus	Baris hilang dari database dan daftar	
7	Input wajib kosong	Ditolak dengan pesan per field	
8	Teks melebihi batas	Ditolak oleh validasi server	

9	ID tidak ada	404 dan tidak membuat data pengganti	
10	Akses tanpa login	Login untuk HTML atau 401 untuk JSON	
11	Teks seperti HTML	Ditampilkan sebagai teks	
12	Tanggal masa depan pada React	Ditolak dan pesan tampil	
13	Interaksi AJAX	Request data tanpa navigasi dokumen penuh	
14	Build React dan TypeScript	Tidak ada error pada pemeriksaan	

Catatan perbaikan _____

_____